INFORMATION MANAGEMENT

OBJECT-ORIENTED PROGRAMMING

PROJECT

Project/System Title: __Transparent and Centralized Grading Management System_____

Group Members: _Oropilla, Condes, Lalata, Mariano, Puyat__

Course / Section: _BSCS - 2A__

Professor: Mrs. Rebecca R. Fajardo_

Project Documentation and Submission Map

Stage	Required Output	Primary Evidence	Professor Remarks
1	Concept Paper	Problem, objectives, scope, users, proposed solution	☐
2	Business Rules	Operational rules, constraints, data relationships	☐
3	ERD	Entities, attributes, PK/FK, cardinality, optionality	☐
4	Normalization	UNF, 1NF, 2NF and 3NF explanation; dependencies and final relations.	☐
5	Data Dictionary	Complete field definitions and constraints	☐
6	MySQL Database	DDL, DML, sample records, queries, screenshots	☐
7	VB.NET System	Forms, CRUD, validation, database connectivity	☐
8	Test Cases	Functional tests, expected/actual results, status	☐
9	User Manual	Installation/access and step-by-step usage guide	☐

General Documentation Standards

☐ All entities and fields shown in the ERD must correspond to the normalized tables and data dictionary.

☐ Primary keys, foreign keys, data types, constraints, and relationships must match the implemented MySQL database.

☐ VB.NET forms and functions must use the actual MySQL database designed in this project.

☐ Use screenshots only as supporting evidence; explain what each screenshot demonstrates.

☐ Use meaningful sample data. Do not use sensitive real-world personal data unless authorized and properly anonymized.

☐ Number and caption major figures/tables (e.g., Figure 1. ERD; Table 3. Data Dictionary).

☐ Attach source code/database scripts as appendices or as separate electronic files when required by the instructor.

1. Concept Paper

The concept paper establishes the need for the proposed system and defines what the group intends to develop.

1.1 Project Title

Transparent and Centralized Grading Management System

1.2 Background / Rationale

Academic grade records maintained in separate spreadsheets can be difficult to consolidate, review and share with authorized users. The group's concept paper identifies delayed awareness of academic standing, overwritten grade information, limited traceability of changes and time-consuming manual interpretation. The system addresses these concerns through centralized records, consistent running-grade calculation, role-based access, revision requests and audit history.

1.3 Problem Statement

Primary Problem: Inefficiencies, vulnerability to calculation errors, and a lack of real-time transparency in traditional manual grade recording and dispute resolution processes.

Supporting Problems:

1. Delays in student visibility regarding individual assessment marks, weighting distributions, and running grades.
2. Absence of an auditable tracking mechanism for grade modifications and formal revision requests.
3. Administrative friction in managing user credentials, class section enrollments, and faculty assignments across academic departments.

1.4 General Objective

To develop a database-driven desktop grading management system for the project context of Pamantasan ng Lungsod ng Pasig that supports current running-grade computation, traceable clarification requests and organized academic record management.

1.5 Specific Objectives

Database Objectives: Implement a relational MySQL/MariaDB schema with keys, referential constraints and parameterized queries. Separate entity data to reduce update anomalies, document the cached running grade, and protect passwords using salted PBKDF2-SHA256 hashing in the active authentication workflow.

System Objectives: Develop a VB.NET Windows Forms application with role-based access control tailored for Students, Teachers, and System Administrators.

Functional Objectives:

- Calculate running grades consistently from raw scores, maximum scores and relative assessment weights, excluding ungraded items and including recorded zero scores.
- Provide student clarification requests and assigned-teacher review with Pending, Under Review, Approved and Closed statuses, recording decisions and approved score corrections.
- Implement automated system audit logs capturing all grade modifications with timestamps and user attributions.

Provide readable PDF grade reports and supported administrative CSV/PDF exports for record review. These are application reports, not a claim of institutional certification.

1.6 Proposed Solution / System Overview

Teachers record raw assessment scores for assigned classes. A shared calculator normalizes relative weights over graded items and updates current standing. Students view their own grades and request clarification. Assigned teachers approve corrections or close requests, with related score and decision changes saved transactionally. Administrators manage accounts, class setup, authorized grade entry and oversight. Audit entries support traceability; they are not cryptographically immutable. Relevant screens refresh when reopened or through periodic database polling.

1.7 Target Users / Stakeholders

User	Role in the system	Main features they use
Students	View their own grades and check how they are doing	Live grade report with running grade and standing, grade history showing who changed a grade and why, revision requests with status tracking, and a performance overview
Professors	Record and manage grades for their classes	Gradebook for entering scores, handling student revision requests, and viewing their assigned classes
Administrators	Manage the system and its records	Student and professor management (add, edit, search, filter), class and subject setup, request monitoring, and audit logs

1.8 Scope

1. Role-based authentication and salted password verification, including compatibility with supported legacy SHA-256 hashes.
2. Score entry and automated dynamic grade recalculation.
3. Database-backed digital gradebook, class roster and assessment CSV imports, and PDF reports.

4. Student grade dispute submission and teacher approval/rejection panel.
5. Class performance analytics dashboard.
6. Comprehensive grade audit logging.
7. Admin account creation and editing, activation/deactivation, protected deletion, class creation and administrative grade entry.

1.9 Limitations

1. Windows desktop interface only; no browser or mobile client is included.
2. No Google Classroom, LMS or university SIS integration is implemented.
3. No student image/video upload or supporting-file attachment feature is implemented.
4. No built-in automated online backup or offline synchronization is implemented.
5. The application requires its configured MySQL/MariaDB database. Local setup is supported; remote access requires network and server configuration. Internet access is not a substitute for database connectivity.

1.10 Significance / Expected Benefits

For Students: Provides access to current recorded scores, standing, history and review responses, supporting earlier awareness of academic performance.

For Faculty: Reduces repeated calculation and data-entry steps and provides structured records of score corrections. Invalid inputs and operational errors remain possible and require validation and review.

For Administration: Consolidates account management and oversight through searchable directories, request monitoring and audit history. These benefits have not been quantified by a formal timing or respondent study in the supplied materials.

1.11 Major System Modules

Module	Main Functions	Primary User/s
Authentication & Session	Role-based login, password verification, active App Session management.	All Users
Teacher Gradebook & Entry	Raw score recording, running grade recalculation, gradebook filter, PDF/CSV export.	Professors
Revision Module	Request submission, review and decisions: Pending, Under Review, Approved and Closed.	Students and Professors
Student/Professor Dashboard	Live grade breakdown, percentage weighting display, score revision history.	Students for student dashboard & Professors for professor dashboard

Analytics & Audit Log	Class averages, top scores, at-risk flags, timestamped audit log of all changes.	Students (own data), Teachers (assigned classes), Admin (oversight)
Admin Management Portal	Student/Professor directory CRUD, class assignments, system stats.	Admin

2. Business Rules

Business rules describe how the organization operates and how data must be stored, related, validated, and processed. Rules should be clear enough to guide the ERD and database constraints.

2.1 Narrative Business Rules

Rule No.	Business Rule	Database/Relationship Impact	Validation/Constraint
1	Each student, teacher and admin must have a unique account ID. Emails must be unique within each role, and passwords must be stored as hashes.	Separate Student, Teacher and Admin tables.	PRIMARY KEY; per-table UNIQUE email; NOT NULL password_hash. Backend: hashing.
2	Only admins may delete accounts without protected related records. Linked accounts may be deactivated instead.	Student.status; Teacher.status; linked records.	Active/Inactive status; restrictive foreign keys. Backend: admin and linked-record checks.
3	A course may have many classes and must not be deleted while classes reference it.	Course 1:M Class.	FOREIGN KEY course_code; ON DELETE RESTRICT.
4	Each class must have one teacher and a unique course, section, semester and academic year combination.	Teacher 1:M Class.	NOT NULL teacher_id foreign key; UNIQUE(course_code, section, semester, academic_year).
5	A student may join many classes but must not enroll in the same class twice.	Student M:N Class through Enrollment.	Foreign keys; UNIQUE(student_id, class_id).
6	Each assessment must belong to one class and term, with a valid type, positive maximum score and positive weight not exceeding 100.	Class 1:M Assessment.	Class foreign key; term/type ENUMs; CHECK maximum > 0, weight 0–100. Backend: weight > 0; two-decimal limits.
7	Each enrollment may have only one grade per assessment.	Enrollment and Assessment linked by StudentGrade.	Foreign keys; UNIQUE(enrollment_id, assessment_id).
8	Scores must remain blank until graded, fall between zero and the maximum, use at most	StudentGrade, Enrollment and Assessment.	Nullable DECIMAL(6,2). Backend: range, precision and same-class checks.

	two decimal places and match the enrollment's class.		
9	Running grades must use relative weights of graded assessments only. Recorded zeros must count; ungraded items must not. Fixed totals or term splits are not required.	Assessment and StudentGrade → Enrollment.running_grade.	Backend: sum(score/max × weight) ÷ sum(graded weights) × 100; NULL if no qualifying grades.
10	A running grade of at least 75% is Passed; below 75% is At risk. No running grade is Not Yet.	Enrollment.running_grade.	Nullable DECIMAL(5,2). Backend: shared calculator and 75% threshold.
11	Students may request a re-check or point increase for their own graded assessments, with only one open request per grade.	StudentGrade 1:M RevisionRequest.	Grade foreign key; reason required; NULL points = re-check. Backend: ownership, positive points and one Pending/Under Review request.
12	Requests must start Pending. The assigned active teacher may mark them Under Review, Approved or Closed. Approval or closure must record reviewer and time. Under Review must be audited. Final decisions must not be processed again.	RevisionRequest; Teacher; AuditLog.	Status ENUM. Backend: transition checks; reviewer/time on approval or closure; audit Under Review.
13	Assigned active teachers and authorized admins may record scores. Only the assigned active teacher may decide revision requests.	Class.teacher_id; Teacher.status; Admin.	Backend: actor authorization, active teacher and class ownership.
14	Important actions must be audited with one actor and a timestamp. Score changes must record available old and new values.	AuditLog references one account actor.	Actor foreign keys; CHECK exactly one actor; default timestamp; optional old/new values.
15	Audit history must remain when its target record is deleted.	AuditLog.entity_type and entity_id.	Entity fields are not foreign keys; INDEX(entity_type, entity_id).
16	Protected references must block deletion. Permitted deletions must follow defined	Class, Enrollment, StudentGrade, RevisionRequest and audit actors.	RESTRICT protected links; CASCADE enrollment/assessment → grades → requests;

	cascade or null-setting rules.		SET NULL reviewed_by.
17	Users must access only role-authorized records and actions. Protected operations must validate applicable role, ownership and account status.	Role-based screens and service operations.	Backend: applicable access and account-status checks.

2.2 Assumptions

1. A class offering is identified by course code, section, semester and academic year. Midterm and Finals are assessment grading periods, distinct from 1st Semester and 2nd Semester.
2. Assessment weights are relative, positive values set for the class; the application does not enforce fixed category percentages, a total of 100 or a 50/50 term split.
3. A committed score correction becomes available to subsequent database reads. Screen refreshes and polling introduce a short display delay; the application does not use push notifications.

2.3 User Roles and Permissions

User Role	Can View	Can Add/Edit/Delete	Special Permissions
Admin	Dashboard, account directories, classes, requests and audit reports.	Create/edit accounts; activate/deactivate; delete eligible unlinked accounts; create classes; enter scores.	Monitor requests read-only; filter and export supported administrative reports.
Student	Own grades, history, performance, profile and request decisions.	Submit own clarification requests; change own password.	Cannot edit scores or decide requests; profile is read-only.
Teacher	Assigned classes, gradebook, requests, analytics and audit records.	Create assessments/classes; enroll active registered students; record scores; decide assigned-class requests; change own password.	CSV imports, PDF reports, fast score entry and surname ordering; no full administrative access.

3. Entity-Relationship Diagram (ERD)

Create the ERD based on the approved business rules. The final ERD must show entities, attributes, primary keys, foreign keys, relationships, cardinality, and optionality where applicable.

☐ All required entities are present.

☐ Every entity has a primary key.

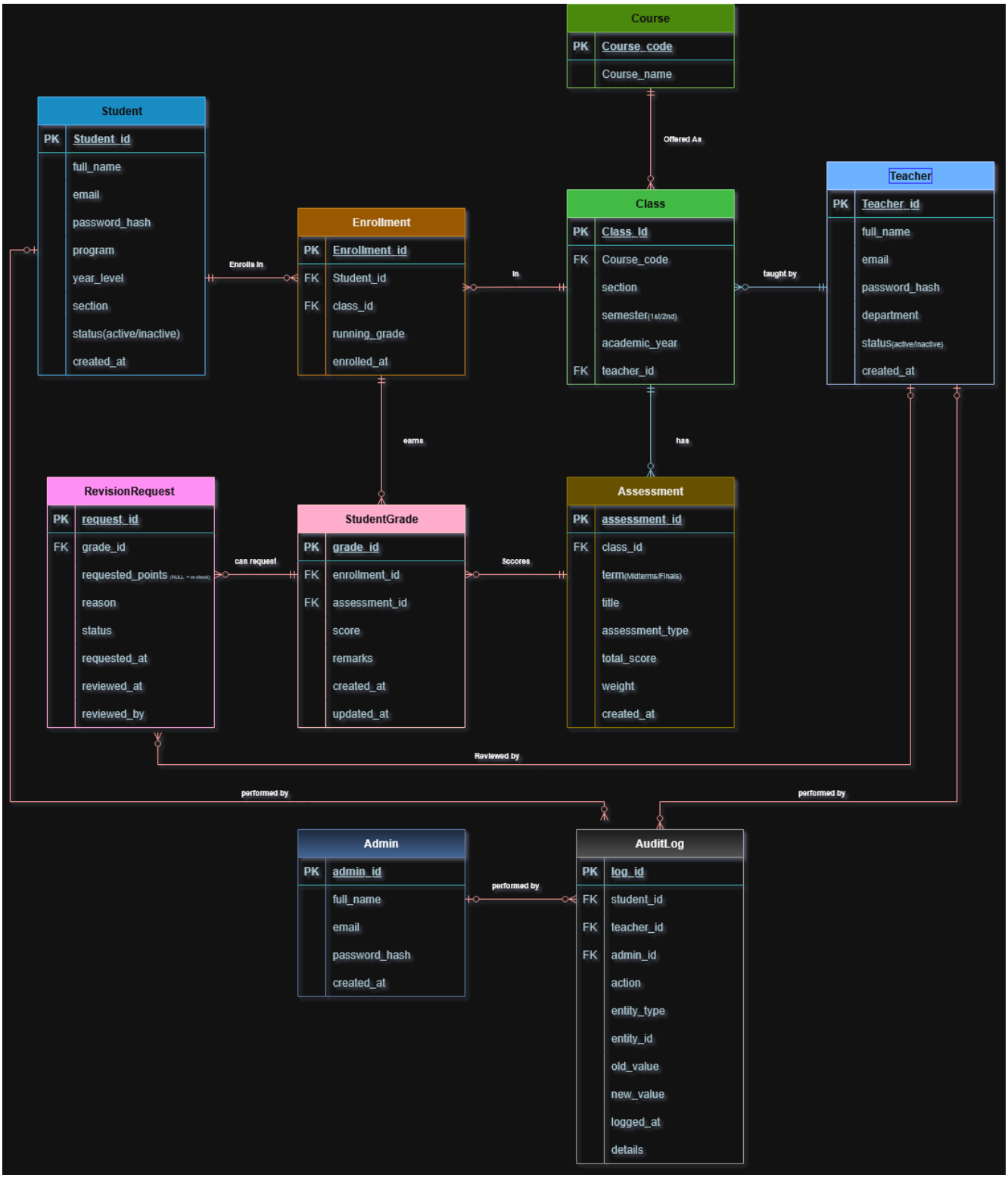
☐ Foreign keys correctly represent relationships.

☐ Many-to-many relationships are resolved using associative/intersection entities.

☐ Attribute names are meaningful and consistent.

☐ Cardinality/optionality agree with the business rules.

3.1 ERD



3.2 ERD Explanation

Student

Stores each student's account and details, such as name, email, program, year level and section. It matters because students are the people the system exists for. They log in to see their grades and to ask for a re-check. It connects to Enrollment, which shows the classes the student takes, and to AuditLog, which records the student's actions.

Teacher

Stores each teacher's account and department. It matters because teachers record the scores and review revision requests. It connects to Class, which shows the classes they handle, to RevisionRequest, which shows the requests they reviewed, and to AuditLog, which records their actions.

Admin

Stores the administrator accounts that manage the system, such as creating accounts and assigning classes. It matters because someone must be accountable for system setup. It connects to AuditLog, which records every admin action.

Course

Stores the subject itself, such as CS 103 and its name. It matters because the course name is stored once here instead of being repeated in every class. This keeps the data clean and follows third normal form. It connects to Class, since one course can be offered as many classes.

Class

Stores one offering of a course: the section, the semester and the academic year. It matters because grades and enrollments belong to a specific class, not just to a subject. It connects to Course (what is taught), Teacher (who teaches it), Enrollment (who is in it) and Assessment (what is graded in it).

Enrollment

Stores which student is in which class, with the running grade for that class. It matters because a student takes many classes and a class has many students. Without it we could not link them. It connects to Student, Class and StudentGrade, which holds the student's scores in that class.

Assessment

Stores each quiz, activity, exam or project, with its term (Midterm or Finals), total score and weight. It matters because it defines what is graded and how much each item counts. It connects to Class (which class it belongs to) and StudentGrade (the scores students got on it).

StudentGrade

Stores one student's score on one assessment, with remarks and timestamps. It matters because it holds the actual grades, which is the main data the system shows. It connects to Enrollment (whose grade it is), Assessment (what was graded) and RevisionRequest (any dispute about the score).

RevisionRequest

Stores a student's request to re-check a score, with the reason, the points asked for and the review status. It matters because it is what makes the system transparent: students can question a grade, and the outcome is recorded. It connects to StudentGrade (the grade in question) and Teacher (who reviewed it).

AuditLog

Stores a permanent record of who did what and when, including the old and new value of a changed score. It matters because it makes every change traceable, so no grade can be changed without a record. It connects to Student, Teacher and Admin, and each entry names exactly one of them. The record that changed is stored as plain text, so the history stays even if that record is deleted later.

4. Database Normalization: 1NF to 3NF

Show the transformation of the initial/un-normalized data structure into First, Second, and Third Normal Form. Do not submit only the final tables; demonstrate the reasoning at each stage.

4.1 Initial / Unnormalized Data

The following worksheets illustrate a design transformation, not an additional production table. A hypothetical unnormalized grade sheet groups a student's classes and repeated assessment/score entries in one record. Repeating assessment columns and duplicated course/teacher details create insertion, update and deletion anomalies.

GRADESHEET

student_id (PK)	student_name	student_email	student_password_hash	program	year_level	student_section	student_status	student_created_at	class_id
25-00418	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active		13
25-00731	Jerome Pagdanganan	jerome.pagdanganan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active		13

course_code	course_name	class_section	semester	academic_year	teacher_id	teacher_name	teacher_email	teacher_password_hash	department
CS 103 COMP 105	Object-Oriented Programming (VB.NET with Database) Information Management	A A	1st Semester 1st Semester	2026-2027 2026-2027	P001 P002	Jhun Alvarez Rebecca R. Fajardo	jhun.alvarez@dauntlesswolves.test rebecca.fajardo@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND REPLACE_WITH_HASH_FROM_BACKEND	CCS CCS
CS 103 COMP 105	Object-Oriented Programming (VB.NET with Database) Information Management	A A	1st Semester 1st Semester	2026-2027 2026-2027	P001 P002	Jhun Alvarez Rebecca R. Fajardo	jhun.alvarez@dauntlesswolves.test rebecca.fajardo@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND REPLACE_WITH_HASH_FROM_BACKEND	CCS CCS

eacher_status	teacher_created_at	enrollment_id	running_grade	enrolled_at	assessment_id	term	title	assessment_type
ctive		1	NULL		1	Midterm	Quiz 1	Quiz
ctive		21	NULL		2	Midterm	Activity 1	Activity
					3	Midterm	Midterm Exam	Exam
					4	Finals	Quiz 2	Quiz
					5	Finals	Project	Project
					6	Finals	Finals Exam	Exam
					---	---	---	---
					13	Midterm	Quiz 1	Quiz
					14	Midterm	Activity 1	Activity
					15	Midterm	Midterm Exam	Exam
					16	Finals	Quiz 2	Quiz
					17	Finals	Project	Project
					18	Finals	Finals Exam	Exam
ctive		2	NULL		1	Midterm	Quiz 1	Quiz
ctive		22	NULL		2	Midterm	Activity 1	Activity
					3	Midterm	Midterm Exam	Exam
					4	Finals	Quiz 2	Quiz

total_score	weight	assessment_created_at	grade_id	score	remarks	grade_created_at	grade_updated_at
20	10		1	14			
50	10		2	41			
100	30		3	82			
20	10		121	NULL			
100	15		122	NULL			
100	25		123	NULL			
---	---	---	---	---	---	---	---
20	10		61	13			
50	10		62	36			
100	30		63	69			

REVISIONREQUEST

request_id (PK)	grade_id	requested_points	reason	request_status	requested_at	reviewed_at	reviewed_by
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001

AUDITLOG

log_id (PK)	action	entity_type	entity_id	old_value	new_value	logged_at	details	actor_student_id	actor_teacher_id	admin_id
1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin			
2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001			2
3	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to P002			2
4	Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts	25-01894		

ADMIN

admin_id (PK)	admin_name	admin_email (AK)	admin_password_hash	admin_created_at		
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND			
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND			
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND			
4	Geoff Andrei Mariano	geoff.mariano@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND			
5	Ron Jacob Puyat	ron.puyat@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND			

4.2 First Normal Form (1NF)

First Normal Form: flatten repeating assessment entries into one row per enrollment–assessment pair and store one value per field. Use (enrollment_id, assessment_id) as the composite candidate key for this illustrative grade relation. Keep independent request and audit histories outside this relation, since each may contain multiple records per grade.

REVISIONREQUEST

request_id (PK)	grade_id (FK)	requested_points	reason	request_status	requested_at	reviewed_at	reviewed_by (FK)
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001
4	67	1	My answer matches the answer key	Closed		2026-10-11 15:30:00	P002

AUDITLOG

log_id (PK)	action	entity_type	entity_id	old_value	new_value	logged_at	details	actor_student_id	actor_teacher_id	admin_id
1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin			
2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001			2
3	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to			2
4	Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts	25-01894		

ADMIN

admin_id (PK)	admin_name	admin_email (AK)	admin_password_hash	admin_created_at
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	

GRADESHEET

student_id (PK)	assessment_id (PK)	grade_id (AK)	enrollment_id (AK)	student_name	student_email	student_password_hash	program	year_level	student_section	student_status
25-00418	1	1	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active
25-00418	2	2	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active
25-00418	3	3	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active
25-00418	4	121	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active
25-00418	5	122	1	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active

student_created_at	class_id	course_code	course_name	class_section	semester	academic_year	teacher_id	teacher_name	teacher_email	teacher_password_hash	department
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS
	1	CS 103	Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS

teacher_status	teacher_created_at	running_grade	enrolled_at	term	title	assessment_type	total_score	weight	assessment_created_at	score	remarks	grade_created_at	grade_updated_at
Active				Midterm	Quiz 1	Quiz	20	10		14			
Active				Midterm	Activity 1	Activity	50	10		41			
Active				Midterm	Midterm Exam	Exam	100	30		82			
Active				Finals	Quiz 2	Quiz	20	10					
Active				Finals	Project	Project	100	15					

4.3 Second Normal Form (2NF)

Second Normal Form: separate attributes that depend on only part of the composite key. Enrollment attributes depend on enrollment_id; assessment definitions depend on assessment_id. The score and grade timestamps depend on the complete enrollment–assessment pair. StudentGrade uses grade_id as its implemented primary key and a UNIQUE constraint on that pair.

STUDENT

student_id (PK)	student_name	student_email (AK)	student_password_hash	program	year_level	student_section	student_status	student_created_at
25-00418	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-00731	Jerome Pagdanganan	jerome.pagdanganan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-01205	Ysabelle Macaraeg	ysabelle.macaraeg@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-01894	Lorenzo Villaflor	lorenzo.villaflor@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	

ASSESSMENT

assessment_id (PK)	term	title	assessment_type	total_score	weight	assessment_created_at	class_id	course_code
1	Midterm	Quiz 1	Quiz	20	10		1	CS 103
2	Midterm	Activity 1	Activity	50	10		1	CS 103
3	Midterm	Midterm Exam	Exam	100	30		1	CS 103
4	Finals	Quiz 2	Quiz	20	10		1	CS 103

course_name	class_section	semester	academic_year	teacher_id	teacher_name	teacher_email	teacher_password_hash	department	teacher_status
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active
Object-Oriented Programming (VB.NET with Database)	A	1st Semester	2026-2027	P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active

teacher_created_at

ENROLLMENT

enrollment_id (PK)	student_id (AK, FK)	class_id (AK)	running_grade	enrolled_at
1	25-00418	1		
2	25-00731	1		
3	25-01205	1		
4	25-01894	1		

REVISIONREQUEST

request_id (PK)	grade_id (FK)	requested_points	reason	request_status	requested_at	reviewed_at	reviewed_by
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001

AUDITLOG

log_id (PK)	action	entity_type	entity_id	old_value	new_value	logged_at	details	actor_student_id	actor_teacher_id	admin_id
1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin			1
2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001			2
3	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to P002			2
4	Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts	25-01894		
5	Submitted revision request	RevisionRequest	2		Pending	2026-10-10 10:30:00	Activity 1: re-check	25-00418		

ENROLLMENT

enrollment_id (PK)	student_id (AK, FK)	class_id (AK, FK)	running_grade	enrolled_at
1	25-00418	1		
2	25-00731	1		
3	25-01205	1		
4	25-01894	1		
5	25-02156	1		

ADMIN

admin_id (PK)	admin_name	admin_email (AK)	admin_password_hash	admin_created_at
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
4	Geoff Andrei Mariano	geoff.mariano@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
5	Ron Jacob Puyat	ron.puyat@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	

STUDENTGRADE

grade_id (PK)	enrollment_id (AK, FK)	assessment_id (AK, FK)	score	remarks	grade_created_at	grade_updated_at
1	1	1	14			
2	1	2	41			
3	1	3	82			
4	2	1	19			

4.4 Third Normal Form (3NF)

Third Normal Form: remove transitive descriptive dependencies. Student details depend on student_id; class details on class_id; course_name on course_code; and teacher details on teacher_id. Store them in Student, Class, Course and Teacher and reference them through keys. Admin, RevisionRequest and AuditLog represent separate entities with their own keys.

Enrollment.running_grade is intentionally cached derived data rather than an independent input. The score-saving transaction maintains it; displays may recompute through the shared calculator. This controlled redundancy must be distinguished from the normalized entity relationships.

STUDENT

student_id (PK)	full_name	email (AK)	password_hash	program	year_level	section	status	created_at
25-00418	Kyla Mae Dimaculangan	kyla.dimaculangan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-00731	Jerome Pagdanganan	jerome.pagdanganan@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	
25-01205	Ysabelle Macaraeg	ysabelle.macaraeg@plp.com	REPLACE_WITH_HASH_FROM_BACKEND	Computer Science	2	A	Active	

TEACHER

teacher_id (PK)	full_name	email (AK)	password_hash	department	status	created_at
P001	Jhun Alvarez	jhun.alvarez@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active	
P002	Rebecca R. Fajardo	rebecca.fajardo@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CCS	Active	
P003	Benito Jose	benito.jose@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	CAS	Active	

ADMIN

admin_id (PK)	full_name	email (AK)	password_hash	created_at
1	Charles Oropilla	charles.oropilla@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
2	Hope Gian Condes	hope.condes@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
3	Carl Jade Lalata	carl.lalata@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
4	Geoff Andrei Mariano	geoff.mariano@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	
5	Ron Jacob Puyat	ron.puyat@dauntlesswolves.test	REPLACE_WITH_HASH_FROM_BACKEND	

COURSE

course_code (PK)	course_name
CS 103	Object-Oriented Programming (VB.NET with Database)
COMP 105	Information Management
CS 12	Software Engineering
GE 008	Ethics

CLASS

class_id (PK)	course_code (AK, FK)	section (AK)	semester (AK)	academic_year (AK)	teacher_id (FK)
1	CS 103	A	1st Semester	2026-2027	P001
2	CS 103	B	1st Semester	2026-2027	P001
3	COMP 105	A	1st Semester	2026-2027	P002
4	COMP 105	B	1st Semester	2026-2027	P002

ASSESSMENT

assessment_id (PK)	class_id (FK)	term	title	assessment_type	total_score	weight	created_at
1	1	Midterm	Quiz 1	Quiz	20	10	
2	1	Midterm	Activity 1	Activity	50	10	
3	1	Midterm	Midterm Exam	Exam	100	30	
4	1	Finals	Quiz 2	Quiz	20	10	
5	1	Finals	Project	Project	100	15	
6	1	Finals	Finals Exam	Exam	100	25	

STUDENTGRADE

grade_id (PK)	enrollment_id (AK, FK)	assessment_id (AK, FK)	score	remarks	created_at	updated_at
1	1	1	14			
2	1	2	41			
3	1	3	82			
4	2	1	19			
5	2	2	34			
6	2	3	93			

REVISIONREQUEST

request_id (PK)	grade_id (FK)	requested_points	reason	status	requested_at	reviewed_at	reviewed_by (FK)
1	10	2	Misread answer on item 5	Pending			
2	2		Please re-check my total score	Under Review			
3	6	5	Partial credit for item 12 was not given	Approved		2026-10-11 15:30:00	P001
4	67	1	My answer matches the answer key	Closed		2026-10-11 15:30:00	P002

AUDITLOG

log_id (PK)	student_id (FK)	teacher_id (FK)	admin_id (FK)	action	entity_type	entity_id	old_value	new_value	logged_at	details
1			1	Created account	Student	25-00418			2026-10-08 09:00:00	Student account created by admin
2			2	Created class	Class	1			2026-10-08 09:10:00	CS 103 Section A assigned to P001
3			2	Created class	Class	3			2026-10-08 09:15:00	COMP 105 Section A assigned to P002
4	25-01894			Submitted revision request	RevisionRequest	1		Pending	2026-10-10 09:15:00	Quiz 1: +2 pts
5	25-00418			Submitted revision request	RevisionRequest	2		Pending	2026-10-10 10:00:00	Activity 1: +5 pts

4.5 Functional Dependency Analysis

Table/Relation	Determinant	Dependent Attribute/s	Action / Normal Form Issue
Grade Sheet (UNORMALIZED)	student_id	classes, assessments, scores	Repeating groups and multi-valued cells (violates 1NF). Flattened to one row per student and assessment.
GRADE_SHEET_1NF	(enrollment_id, assessment_id)	all other columns	Minimal candidate key. 1NF satisfied: atomic values, unique rows.
GRADE_SHEET_1NF	student_id	student name, email, password_hash, program, year_level, section, status, created_at	Transitive descriptive dependency through Enrollment; isolate Student details in 3NF.
GRADE_SHEET_1NF	assessment_id	class_id, term, title, assessment_type, total_score, weight, created_at, and the class, course and teacher details	Partial dependency (2NF). Moved to Assessment.
GRADE_SHEET_1NF	enrollment_id	student_id, class_id, running_grade, enrolled_at	Partial dependency (2NF). Moved to Enrollment.
GRADE_SHEET_1NF	(enrollment_id, assessment_id)	grade_id, score, remarks, created_at, updated_at	Full dependency. Kept as StudentGrade.
Assessment (2NF)	class_id	course_code, section, semester, academic_year, teacher_id	Transitive dependency (assessment_id → class_id → class details). Moved to Class (3NF).
Class	course_code	course_name	Transitive dependency (class_id → course_code → course_name). Moved to Course (3NF).
Class	teacher_id	teacher name, email, password_hash, department, status, created_at	Transitive dependency (class_id → teacher_id → teacher details). Moved to Teacher (3NF).
Final 10 tables (see 4.6)	Each table's primary key	All other attributes of that table	Entity relations are separated by their determinants. running_grade is intentionally cached derived data maintained by the application.

4.6 Final 3NF Tables

Table Name	Primary Key	Foreign Key/s	Major Non-Key Attributes	Purpose
Student	student_id	None	full_name, email, password_hash,	Stores student accounts. Students

			program, year_level, section, status, created_at	log in to view their grades and send revision requests.
Teacher	teacher_id	None	full_name, email, password_hash, department, employment_type, status, created_at	Stores teacher accounts. Teachers handle classes, record scores and review revision requests.
Admin	admin_id	None	full_name, email, password_hash, created_at	Stores admin accounts. Admins manage student and teacher accounts and class assignments.
Course	course_code	None	course_name	Stores the subjects offered, such as CS 103 Object-Oriented Programming.
Class	class_id	course_code (Course), teacher_id (Teacher)	section, semester, academic_year	One offering of a course for a section, semester and academic year, handled by one teacher.
Enrollment	enrollment_id	student_id (Student), class_id (Class)	running_grade, enrolled_at	Places a student in a class. Resolves the many-to-many between Student and Class. A student can enroll in a class only once.
Assessment	assessment_id	class_id (Class)	term, title, assessment_type, total_score, weight, created_at	A gradable item in a class (quiz, activity, exam, project) with its term and weight.
StudentGrade	grade_id	enrollment_id (Enrollment), assessment_id (Assessment)	score, remarks, created_at, updated_at	One student's score on one assessment. Resolves the many-to-many between Enrollment and Assessment. One score per student per assessment.
RevisionRequest	request_id	grade_id (StudentGrade),	requested_points, reason, status,	Student score concern and decision;

		reviewed_by (Teacher)	requested_at, reviewed_at	reviewer/time are populated on approval or closure, not merely Under Review.
AuditLog	log_id	student_id (Student), teacher_id (Teacher), admin_id (Admin)	action, entity_type, entity_id, old_value, new_value, logged_at, details	Permanent record of who did what. Exactly one actor ID is filled per entry.

5. Data Dictionary

Document every field in every final 3NF table. The data dictionary must match the actual MySQL implementation.

Table	Field	Data Type	Size	PK/FK	Null?	Default	Constraint	Description
Student	student_id	VARCHAR	10	PK	NO	-	PRIMARY KEY	Student ID number (e.g. 25-06767)
Student	full_name	VARCHAR	100	-	NO	-	-	Student's full name
Student	email	VARCHAR	100	-	NO	-	UNIQUE	School email address
Student	password_hash	VARCHAR	255	-	NO	-	-	Hashed login password
Student	program	VARCHAR	100	-	NO	-	-	Degree program (e.g. BS Computer Science)
Student	year_level	TINYINT UNSIGNED	-	-	NO	-	-	Year level of the student
Student	section	VARCHAR	20	-	NO	-	-	Section (e.g. BSCS 2A)

Student	status	ENUM	-	-	NO	‘Active’	Values: Active, Inactive	Whether the account is in use
Student	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
Teacher	teacher_id	VARCHAR	10	PK	NO	-	PRIMARY KEY	Teacher ID (e.g. P001)
Teacher	full_name	VARCHAR	100	-	NO	-	-	Teacher's full name
Teacher	email	VARCHAR	100	-	NO	-	UNIQUE	School email address
Teacher	password_hash	VARCHAR	255	-	NO	-	-	Hashed login password
Teacher	department	VARCHAR	50	-	NO	-	-	Department the teacher belongs to
Teacher	employment_type	ENUM	-	-	NO	Full-time	Full-time, Part-time	Teacher employment classification
Teacher	status	ENUM	-	-	NO	‘Active’	Values: Active, Inactive	Whether the account is in use
Teacher	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
Admin	admin_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_I	Admin identifier

							NCREMENT	
Admin	full_name	VARCHAR	100	-	NO	-	-	Admin's full name
Admin	email	VARCHAR	100	-	NO	-	UNIQUE	Admin email address
Admin	password_hash	VARCHAR	255	-	NO	-	-	Hashed login password
Admin	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
Course	course_code	VARCHAR	20	PK	NO	-	PRIMARY KEY	Course code (e.g. CS 103)
Course	course_name	VARCHAR	100	-	NO	-	-	Course title (e.g. Object Oriented Programming)
Class	class_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Class identifier
Class	course_code	VARCHAR	20	FK	NO	-	FOREIGN KEY →Course(course_code), ON UPDATE CASCADE, ON DELETE RESTRICT;part of	Course this class belongs to

							UNIQUE uq_class	
Class	section	VARCHAR	20	-	NO	-	Part of UNIQUE uq_class	Section (e.g. BSCS 2A)
Class	semester	ENUM	-	-	NO	-	Values:1st Semester, 2nd Semester; part of UNIQUE uq_class	Semester the class runs in
Class	academic_year	VARCHAR	9	-	NO	-	Part of UNIQUE uq_class	Academic year (e.g. 2026-2027)
Class	teacher_id	VARCHAR	10	FK	NO	-	FOREIGN KEY →Teacher (teacher_id),ON UPDATE CASCADE, ON DELETE RESTRICT	Teacher handling the class
Enrollment	enrollment_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Enrollment identifier
Enrollment	student_id	VARCHAR	10	FK	NO	-	FOREIGN KEY →Student (student_id), ON UPDATE CASCA	Enrolled student

							DE, ON DELETE RESTRICT; part of UNIQUEuq_enrollment	
Enrollment	class_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY →Class (class_id), ON UPDATE CASCADE, ON DELETE RESTRICT; part of UNIQUEuq_enrollment	Class the student is enrolled in
Enrollment	running_grade	DECIMAL	5,2	-	YES	NULL	-	Overall standing in the class; NULL until the first score is recorded
Enrollment	enrolled_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the student was enrolled
Assessment	assessment_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Assessment identifier

Assessment	class_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → Class(class_id), ON UPDATE CASCADE, ON DELETE CASCADE	Class the assessment belongs to
Assessment	term	ENUM	-	-	NO	-	Values: Midterm, Finals	Grading period
Assessment	title	VARCHAR	100	-	NO	-	-	Assessment name (e.g. Quiz 2)
Assessment	assessment_type	ENUM	-	-	NO	-	Values: Quiz, Activity, Exam, Project, Attendance	Kind of assessment
Assessment	total_score	DECIMAL	6,2	-	NO	-	CHECK (total_score > 0)	Highest possible score
Assessment	weight	DECIMAL	5,2	-	NO	-	CHECK (weight BETWEEN 0 AND 100)	Relative assessment weight; not an enforced category percentage

Assessment	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the record was created
StudentGrade	grade_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Grade identifier
StudentGrade	enrollment_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → Enrollment(enrollment_id), ON UPDATECASCADE, ON DELETECASCADE; part of UNIQUE uq_grade	Enrollment the grade belongs to
StudentGrade	assessment_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → Assessment(assessment_id), ON UPDATECASCADE, ON DELETECASCADE; part of UNIQUE uq_grade	Assessment being graded

StudentGrade	score	DECIMAL	6,2	-	YES	NULL	-	Score earned; NULL until graded
StudentGrade	remarks	VARCHAR	255	-	YES	NULL	-	Stored assessment remarks, maintained by score-saving logic
StudentGrade	created_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the grade row was created
StudentGrade	updated_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	When the grade row was last changed
RevisionRequest	request_id	INT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Request identifier
RevisionRequest	grade_id	INT UNSIGNED	-	FK	NO	-	FOREIGN KEY → StudentGrade(grad_id), ON UPDATE CASCADE, ON DELETE CASCADE	Grade being questioned

RevisionRequest	requested_points	DECIMAL	5,2	-	YES	NULL	-	Points asked for (e.g. 2.00); NULL for a plain re-check
RevisionRequest	reason	VARCHAR	255	-	NO	-	-	Student's reason for the request
RevisionRequest	status	ENUM	-	-	NO	'Pending'	Values: Pending, Under Review, Approved ,Closed	Where the request stands
RevisionRequest	requested_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the request was filled
RevisionRequest	reviewed_at	TIMESTAMP	-	-	YES	NULL	-	Approval or closure time; NULL before a final decision
RevisionRequest	reviewed_by	VARCHAR	10	FK	YES	NULL	FOREIGN KEY → Teacher(teacher_id), ON UPDATE CASCADE, ON DELETE SET NULL	Teacher who approved or closed the request; nullable
AuditLog	log_id	BIGINT UNSIGNED	-	PK	NO	-	PRIMARY KEY, AUTO_INCREMENT	Log entry identifier

AuditLog	student_id	VARCHAR	10	FK	YES	NULL	FOREIGN KEY → Student(student_id), ON DELETE RESTRICT; CHECK exactly one actor	Student who did the action
AuditLog	teacher_id	VARCHAR	10	FK	YES	NULL	FOREIGN KEY → Teacher(teacher_id), ON DELETE RESTRICT; CHECK exactly one actor	Teacher who did the action
AuditLog	admin_id	INT UNSIGNED	-	FK	YES	NULL	FOREIGN KEY → Admin(admin_id), ON DELETE RESTRICT; CHECK exactly one actor	Admin who did the action
AuditLog	action	VARCHAR	100	-	NO	-	-	What was done (e.g. Grade updated)

AuditLog	entity_type	VARCHAR	50	-	NO	-	INDEX idx_entity	Kind of record affected (not a foreign key)
AuditLog	entity_id	VARCHAR	20	-	NO	-	INDEX idx_entity	ID of the record affected (not a foreign key)
AuditLog	old_value	VARCHAR	100	-	YES	NULL	-	Value before a score change (e.g. 16/20)
AuditLog	new_value	VARCHAR	100	-	YES	NULL	-	Value after a score change (e.g. 18/20)
AuditLog	logged_at	TIMESTAMP	-	-	NO	CURRENT_TIMESTAMP	-	When the action was recorded
AuditLog	details	VARCHAR	500	-	YES	NULL	-	Extra notes about the action

6. MySQL Database Implementation

Implement the approved 3NF design in MySQL/MariaDB. The implementation must be consistent with the ERD and data dictionary.

6.1 Database Creation

Database name used in the supplied SQL dump: centralized_and_transparent_grading_system.

CREATE DATABASE IF NOT EXISTS centralized_and_transparent_grading_system CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE centralized_and_transparent_grading_system;

The Database value in dbconfig.json must exactly match the imported database. The application fallback name is different, so explicitly configure it rather than relying on defaults.

```
MariaDB [(none)]> create database centralized_and_transparent_grading_system
    -> character set utf8mb4
    -> collate utf8mb4_unicode_ci;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]>
```

6.2 DDL – Table Creation

The screenshots below document the ten implemented tables. Use the supplied SQL dump as the executable source for exact definitions, constraints and sample rows. It contains teacher.employment_type, which is also included in the completed data dictionary. Database foreign keys enforce relationships; cross-table score limits, class ownership and request transitions are checked by the application.

```
MariaDB [centralized_and_transparent_grading_system]> create table Student(
    -> student_id varchar(10) primary key,
    -> full_name varchar(100) not null,
    -> email varchar(100) not null unique,
    -> password_hash varchar(255) not null,
    -> program varchar(100) not null,
    -> year_level tinyint unsigned not null,
    -> section varchar(20) not null,
    -> status enum('Active','Inactive
    '> ') not null default 'Active',
    -> created_at timestamp not null default CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.076 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> alter table Student
    -> modify column status ENUM('Active', 'Inactive')
    -> not null default 'Active';
Query OK, 21 rows affected (0.081 sec)
Records: 21  Duplicates: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Teacher(
    -> teacher_id varchar(10) primary key,
    -> full_name varchar(100) not null,
    -> email varchar(100) not null unique,
    -> password_hash varchar(255) not null,
    -> department varchar(50) not null,
    -> status enum('Active','Inactive') not null default 'Active',
    -> created_at timestamp not null default CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.064 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Admin(
    -> admin_id int unsigned primary key auto_increment,
    -> full_name varchar(100) not null,
    -> email varchar(100) not null unique,
    -> password_hash varchar(255) not null,
    -> created_at timestamp not null default CURRENT_TIMESTAMP);
Query OK, 0 rows affected (0.037 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Course(
    -> course_code varchar(20) primary key,
    -> course_name varchar(100) not null);
Query OK, 0 rows affected (0.012 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Class(
    -> class_id int unsigned primary key auto_increment,
    -> course_code varchar(20) not null,
    -> section varchar(20) not null,
    -> semester enum('1st Semester', '2nd Semester') not null,
    -> academic_year varchar(9) not null,
    -> teacher_id varchar(10) not null,
    -> constraint fk_class_course
    -> Foreign Key(course_code) references Course(course_code)
    -> on update cascade on delete restrict,
    -> constraint fk_class_teacher
    -> Foreign Key(teacher_id) references Teacher(teacher_id)
    -> on update cascade on delete restrict,
    -> unique key uq_class (course_code, section, semester, academic_year));
Query OK, 0 rows affected (0.083 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Enrollment(
    -> enrollment_id int unsigned primary key auto_increment,
    -> student_id varchar(10) not null,
    -> class_id int unsigned not null,
    -> running_grade decimal(5,2) null,
    -> enrolled_at timestamp not null default CURRENT_TIMESTAMP,
    -> constraint fk_enrollment_student
    -> foreign key(student_id) references Student(student_id)
    -> on update cascade on delete restrict,
    -> constraint fk_enrollment_class
    -> foreign key(class_id) references Class(class_id)
    -> on update cascade on delete restrict,
    -> unique key uq_enrollment(student_id, class_id));
Query OK, 0 rows affected (0.075 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table Assessment(
    -> assessment_id int unsigned primary key auto_increment,
    -> class_id int unsigned not null,
    -> term enum('Midterm','Finals') not null,
    -> title varchar(100) not null,
    -> assessment_type enum('Quiz','Activity','Exam','Project','Attendance') not null,
    -> total_score decimal(6,2) not null,
    -> weight decimal(5,2) not null,
    -> created_at timestamp not null default CURRENT_TIMESTAMP,
    -> constraint fk_assessment_class
    -> foreign key(class_id) references Class(class_id)
    -> on update cascade on delete cascade,
    -> constraint chk_weight check(weight BETWEEN 0 AND 100),
    -> constraint chk_total_score check(total_score > 0));
Query OK, 0 rows affected (0.070 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table StudentGrade(
    -> grade_id int unsigned primary key auto_increment,
    -> enrollment_id int unsigned not null,
    -> assessment_id int unsigned not null,
    -> score decimal(6,2) null,
    -> remarks varchar(255) null,
    -> created_at timestamp not null default CURRENT_TIMESTAMP,
    -> updated_at timestamp not null default CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    -> constraint fk_grade_enrollment
    -> foreign key(enrollment_id) references Enrollment(enrollment_id)
    -> on update cascade on delete cascade,
    -> constraint fk_grade_assessment
    -> foreign key(assessment_id) references Assessment(assessment_id)
    -> on update cascade on delete cascade,
    -> unique key uq_grade(enrollment_id, assessment_id));
Query OK, 0 rows affected (0.036 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table RevisionRequest(
    -> request_id int unsigned primary key auto_increment,
    -> grade_id int unsigned not null,
    -> requested_points decimal(5,2) null,
    -> reason varchar(255) not null,
    -> status enum('Pending','Under Review','Approved','Closed') not null default 'Pending',
    -> requested_at timestamp not null default CURRENT_TIMESTAMP,
    -> reviewed_at timestamp null,
    -> reviewed_by varchar(10) null,
    -> constraint fk_request_grade
    -> foreign key(grade_id) references StudentGrade(grade_id)
    -> on update cascade on delete cascade,
    -> constraint fk_request_teacher
    -> foreign key(reviewed_by) references Teacher(teacher_id)
    -> on update cascade on delete set null);
Query OK, 0 rows affected (0.082 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> create table AuditLog(
    -> log_id bigint unsigned primary key auto_increment,
    -> student_id varchar(10) null,
    -> teacher_id varchar(10) null,
    -> admin_id int unsigned null,
    -> action varchar(100) not null,
    -> entity_type varchar(50) not null,
    -> entity_id varchar(20) not null,
    -> old_value varchar(100) null,
    -> new_value varchar(100) null,
    -> logged_at timestamp not null default CURRENT_TIMESTAMP,
    -> details varchar(500) null,
    -> constraint fk_audit_student
    -> foreign key(student_id) references Student(student_id)
    -> on delete restrict,
    -> constraint fk_audit_teacher
    -> foreign key(teacher_id) references Teacher(teacher_id)
    -> on delete restrict,
    -> constraint fk_audit_admin
    -> foreign key(admin_id) references Admin(admin_id)
    -> on delete restrict,
    -> constraint chk_audit_one_actor
    -> check((student_id is not null) + (teacher_id is not null) + (admin_id is not null) = 1),
    -> index idx_entity (entity_type, entity_id));
Query OK, 0 rows affected (0.069 sec)
```

6.3 DML – Sample Data

Import the supplied SQL dump into a disposable or project database using MySQL/MariaDB tools. It includes fictional sample accounts, class offerings, enrollments, assessments, scores, requests and audits. Some seeded passwords are placeholders and must be initialized through an approved account setup procedure before login; do not use placeholder hash text as a password. CSV imports require registered active students and do not create accounts.

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Admin.csv'
    -> INTO TABLE Admin
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (full_name, email, password_hash);
Query OK, 5 rows affected (0.039 sec)
Records: 5  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Teacher.csv'
    -> INTO TABLE Teacher
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (teacher_id, full_name, email, password_hash, department);
Query OK, 3 rows affected (0.044 sec)
Records: 3  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Student.csv'
    -> INTO TABLE Student
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (student_id, full_name, email, password_hash, program, year_level, section);
Query OK, 21 rows affected (0.041 sec)
Records: 21  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Course.csv'
    -> INTO TABLE Course
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (course_code, course_name);
Query OK, 4 rows affected (0.054 sec)
Records: 4  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Class.csv'
    -> INTO TABLE Class
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (course_code, section, semester, academic_year, teacher_id);
Query OK, 4 rows affected (0.052 sec)
Records: 4  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Enrollment.csv'
    -> INTO TABLE Enrollment
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (student_id, class_id);
Query OK, 40 rows affected (0.044 sec)
Records: 40  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/Assessment.csv'
    -> INTO TABLE Assessment
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (class_id, term, title, assessment_type, total_score, weight);
Query OK, 24 rows affected (0.045 sec)
Records: 24  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/StudentGrade.csv'
    -> INTO TABLE StudentGrade
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (enrollment_id, assessment_id, score);
Query OK, 120 rows affected (0.050 sec)
Records: 120  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> insert into StudentGrade(enrollment_id, assessment_id, score)
    -> select
    -> e.enrollment_id,
    -> a.assessment_id,
    -> null
    -> FROM Enrollment e
    -> JOIN Assessment a
    -> ON a.class_id = e.class_id
    -> WHERE a.term = 'Finals'
    -> AND NOT EXISTS(
    -> SELECT 1
    -> FROM StudentGrade sg
    -> WHERE sg.enrollment_id = e.enrollment_id
    -> AND sg.assessment_id = a.assessment_id);
Query OK, 120 rows affected (0.047 sec)
Records: 120  Duplicates: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/RevisionRequest.cs
v'
    -> INTO TABLE RevisionRequest
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (grade_id, requested_points, reason, status, reviewed_at, reviewed_by);
Query OK, 4 rows affected (0.043 sec)
Records: 4  Deleted: 0  Skipped: 0  Warnings: 0
```

```
MariaDB [centralized_and_transparent_grading_system]> LOAD DATA LOCAL INFILE 'C:/Users/Mely Rose/Desktop/for import sa grading system/AuditLog.csv'
    -> INTO TABLE AuditLog
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\r\n'
    -> IGNORE 1 LINES
    -> (student_id, teacher_id, admin_id, action, entity_type, entity_id, old_value, new_value, logged_at, details);
Query OK, 11 rows affected (0.048 sec)
Records: 11  Deleted: 0  Skipped: 0  Warnings: 0
```

6.4 Required Queries

Examples below supplement the existing query screenshots. Replace parameter values through bound parameters in application code.

SELECT s.student_id, s.full_name, c.course_code, c.section FROM enrollment e JOIN student s ON s.student_id=e.student_id JOIN class c ON c.class_id=e.class_id WHERE e.class_id=@ClassId ORDER BY s.full_name;

SELECT e.enrollment_id, ROUND(SUM(sg.score/a.total_score*a.weight)/SUM(a.weight)*100,2) AS running_grade FROM enrollment e JOIN studentgrade sg ON sg.enrollment_id=e.enrollment_id JOIN assessment a ON a.assessment_id=sg.assessment_id AND a.class_id=e.class_id WHERE e.class_id=@ClassId AND sg.score IS NOT NULL AND a.total_score>0 AND a.weight>0 GROUP BY e.enrollment_id HAVING running_grade<75;

SELECT r.request_id,r.status,r.requested_at,s.full_name,a.title FROM revisionrequest r JOIN studentgrade sg ON sg.grade_id=r.grade_id JOIN enrollment e ON e.enrollment_id=sg.enrollment_id JOIN student s ON s.student_id=e.student_id JOIN assessment a ON a.assessment_id=sg.assessment_id WHERE r.status IN ('Pending','Under Review') ORDER BY r.requested_at DESC;

```
MariaDB [centralized_and_transparent_grading_system]> select student_id, full_name, email
    -> from student
    -> where program = 'Computer Science'
    -> AND year_level = 2
    -> AND section = 'A';
ERROR 2006 (HY000): MySQL server has gone away
No connection. Trying to reconnect...
Connection id:    37
Current database: centralized_and_transparent_grading_system

+------------+--------------------------+------------------------------+
| student_id | full_name                | email                        |
+------------+--------------------------+------------------------------+
| 25-00418   | Kyla Mae Dimaculangan    | kyla.dimaculangan@plp.com    |
| 25-00731   | Jerome Pagdanganan       | jerome.pagdanganan@plp.com   |
| 25-01205   | Ysabelle Macaraeg        | ysabelle.macaraeg@plp.com    |
| 25-01894   | Lorenzo Villaflor        | lorenzo.villaflor@plp.com    |
| 25-02156   | Trisha Anne Alcantara    | trisha.alcantara@plp.com     |
| 25-02643   | Nathaniel Evangelista    | nathaniel.evangelista@plp.com |
| 25-03087   | Princess Joy Magbanua    | princess.magbanua@plp.com    |
| 25-03519   | Miguel Andre Panganiban  | miguel.panganiban@plp.com    |
| 25-04072   | Samantha Rose Sarmiento  | samantha.sarmiento@plp.com   |
| 25-04396   | Gabriel Zandro Natividad | gabriel.natividad@plp.com    |
| 25-05128   | Jasmine Faye Catapang    | jasmine.catapang@plp.com     |
| 25-05764   | Elijah Rebolledo         | elijah.rebolledo@plp.com     |
| 25-06291   | Daniella Abellera        | daniella.abellera@plp.com    |
+------------+--------------------------+------------------------------+
13 rows in set (0.082 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> select class_id, COUNT(*) AS enrolled_students
    -> FROM Enrollment
    -> GROUP BY class_id
    -> ORDER BY class_id;
+----------+-------------------+
| class_id | enrolled_students |
+----------+-------------------+
|        1 |                12 |
|        2 |                 8 |
|        3 |                12 |
|        4 |                 8 |
+----------+-------------------+
4 rows in set (0.024 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> select class_id, COUNT(*) AS enrolled_students
    -> FROM Enrollment
    -> GROUP BY class_id
    -> HAVING COUNT(*) > 10
    -> ORDER BY enrolled_students DESC;
+----------+-------------------+
| class_id | enrolled_students |
+----------+-------------------+
|        1 |                12 |
|        3 |                12 |
+----------+-------------------+
2 rows in set (0.012 sec)
```

6.5 Database Evidence

```
MariaDB [centralized_and_transparent_grading_system]> show tables;
+-----------------------------------------------------------+
| Tables_in_centralized_and_transparent_grading_system |
+-----------------------------------------------------------+
| admin                                                     |
| assessment                                                |
| auditlog                                                  |
| class                                                     |
| course                                                    |
| enrollment                                                |
| revisionrequest                                           |
| student                                                   |
| studentgrade                                              |
| teacher                                                   |
+-----------------------------------------------------------+
10 rows in set (0.031 sec)

MariaDB [centralized_and_transparent_grading_system]>
```

```
MariaDB [centralized_and_transparent_grading_system]> select * from student
    -> where section = 'A'
    -> limit 7;
+------------+---------------------+---------------------------+------------------------------+------------------+------------+---------+--------+---------------------+
| student_id | full_name           | email                     | password_hash                | program          | year_level | section | status | created_at          |
+------------+---------------------+---------------------------+------------------------------+------------------+------------+---------+--------+---------------------+
| 25-00418   | Kyla Mae Dimaculangan | kyla.dimaculangan@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-00731   | Jerome Pagdanganan  | jerome.pagdanganan@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-01205   | Ysabelle Macaraeg   | ysabelle.macaraeg@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-01894   | Lorenzo Villaflor   | lorenzo.villaflor@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-02156   | Trisha Anne Alcantara | trisha.alcantara@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-02643   | Nathaniel Evangelista | nathaniel.evangelista@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
| 25-03087   | Princess Joy Magbanua | princess.magbanua@plp.com | REPLACE_WITH_HASH_FROM_BACKEND | Computer Science |          2 | A       | Active | 2026-10-14 21:46:54 |
+------------+---------------------+---------------------------+------------------------------+------------------+------------+---------+--------+---------------------+
7 rows in set (0.012 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> select * from teacher;
+------------+--------------------+-----------------------------------+-------------------------------+------------+--------+---------------------+
| teacher_id | full_name          | email                             | password_hash                 | department | status | created_at          |
+------------+--------------------+-----------------------------------+-------------------------------+------------+--------+---------------------+
| P001       | Jhun Alvarez       | jhun.alvarez@dauntlesswolves.test | REPLACE_WITH_HASH_FROM_BACKEND | CCS        | Active | 2026-10-14 21:44:38 |
| P002       | Rebecca R. Fajardo | rebecca.fajardo@dauntlesswolves.test | REPLACE_WITH_HASH_FROM_BACKEND | CCS        | Active | 2026-10-14 21:44:38 |
| P003       | Benito Jose        | benito.jose@dauntlesswolves.test  | REPLACE_WITH_HASH_FROM_BACKEND | CAS        | Active | 2026-10-14 21:44:38 |
+------------+--------------------+-----------------------------------+-------------------------------+------------+--------+---------------------+
3 rows in set (0.008 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> SELECT * FROM vw_class_grade_sheet
    -> WHERE class_id = 1 AND term = 'Midterm'
    -> ORDER BY full_name, assessment_id;
+----------+------------+-------------------------+---------------+---------+--------------+---------+-------------+--------+
| class_id | student_id | full_name               | assessment_id | term    | title        | score   | total_score | weight |
+----------+------------+-------------------------+---------------+---------+--------------+---------+-------------+--------+
|        1 | 25-05764   | Elijah Rebolledo        |             1 | Midterm | Quiz 1       |  16.00  |       20.00 |  10.00 |
|        1 | 25-05764   | Elijah Rebolledo        |             2 | Midterm | Activity 1   |  34.00  |       50.00 |  10.00 |
|        1 | 25-05764   | Elijah Rebolledo        |             3 | Midterm | Midterm Exam |  76.00  |      100.00 |  30.00 |
|        1 | 25-04396   | Gabriel Zandro Natividad |             1 | Midterm | Quiz 1       |  14.00  |       20.00 |  10.00 |
|        1 | 25-04396   | Gabriel Zandro Natividad |             2 | Midterm | Activity 1   |  37.00  |       50.00 |  10.00 |
|        1 | 25-04396   | Gabriel Zandro Natividad |             3 | Midterm | Midterm Exam |  97.00  |      100.00 |  30.00 |
|        1 | 25-05128   | Jasmine Faye Catapang   |             1 | Midterm | Quiz 1       |  19.00  |       20.00 |  10.00 |
|        1 | 25-05128   | Jasmine Faye Catapang   |             2 | Midterm | Activity 1   |  46.00  |       50.00 |  10.00 |
|        1 | 25-05128   | Jasmine Faye Catapang   |             3 | Midterm | Midterm Exam |  98.00  |      100.00 |  30.00 |
|        1 | 25-00731   | Jerome Pagdanganan      |             1 | Midterm | Quiz 1       |  19.00  |       20.00 |  10.00 |
|        1 | 25-00731   | Jerome Pagdanganan      |             2 | Midterm | Activity 1   |  34.00  |       50.00 |  10.00 |
|        1 | 25-00731   | Jerome Pagdanganan      |             3 | Midterm | Midterm Exam |  93.00  |      100.00 |  30.00 |
|        1 | 25-00418   | Kyla Mae Dimaculangan   |             1 | Midterm | Quiz 1       |  14.00  |       20.00 |  10.00 |
|        1 | 25-00418   | Kyla Mae Dimaculangan   |             2 | Midterm | Activity 1   |  41.00  |       50.00 |  10.00 |
|        1 | 25-00418   | Kyla Mae Dimaculangan   |             3 | Midterm | Midterm Exam |  82.00  |      100.00 |  30.00 |
|        1 | 25-01894   | Lorenzo Villaflor       |             1 | Midterm | Quiz 1       |   9.00  |       20.00 |  10.00 |
|        1 | 25-01894   | Lorenzo Villaflor       |             2 | Midterm | Activity 1   |  23.00  |       50.00 |  10.00 |
|        1 | 25-01894   | Lorenzo Villaflor       |             3 | Midterm | Midterm Exam |  45.00  |      100.00 |  30.00 |
|        1 | 25-03519   | Miguel Andre Panganiban |             1 | Midterm | Quiz 1       |  14.00  |       20.00 |  10.00 |
|        1 | 25-03519   | Miguel Andre Panganiban |             2 | Midterm | Activity 1   |  46.00  |       50.00 |  10.00 |
|        1 | 25-03519   | Miguel Andre Panganiban |             3 | Midterm | Midterm Exam |  80.00  |      100.00 |  30.00 |
|        1 | 25-02643   | Nathaniel Evangelista   |             1 | Midterm | Quiz 1       |  18.00  |       20.00 |  10.00 |
|        1 | 25-02643   | Nathaniel Evangelista   |             2 | Midterm | Activity 1   |  45.00  |       50.00 |  10.00 |
|        1 | 25-02643   | Nathaniel Evangelista   |             3 | Midterm | Midterm Exam |  90.00  |      100.00 |  30.00 |
|        1 | 25-03087   | Princess Joy Magbanua   |             1 | Midterm | Quiz 1       |  17.00  |       20.00 |  10.00 |
|        1 | 25-03087   | Princess Joy Magbanua   |             2 | Midterm | Activity 1   |  36.00  |       50.00 |  10.00 |
|        1 | 25-03087   | Princess Joy Magbanua   |             3 | Midterm | Midterm Exam |  85.00  |      100.00 |  30.00 |
|        1 | 25-04072   | Samantha Rose Sarmiento |             1 | Midterm | Quiz 1       |  18.00  |       20.00 |  10.00 |
|        1 | 25-04072   | Samantha Rose Sarmiento |             2 | Midterm | Activity 1   |  44.00  |       50.00 |  10.00 |
|        1 | 25-04072   | Samantha Rose Sarmiento |             3 | Midterm | Midterm Exam |  87.00  |      100.00 |  30.00 |
|        1 | 25-02156   | Trisha Anne Alcantara   |             1 | Midterm | Quiz 1       |  17.00  |       20.00 |  10.00 |
|        1 | 25-02156   | Trisha Anne Alcantara   |             2 | Midterm | Activity 1   |  45.00  |       50.00 |  10.00 |
|        1 | 25-02156   | Trisha Anne Alcantara   |             3 | Midterm | Midterm Exam |  83.00  |      100.00 |  30.00 |
|        1 | 25-01205   | Ysabelle Macaraeg       |             1 | Midterm | Quiz 1       |  17.00  |       20.00 |  10.00 |
|        1 | 25-01205   | Ysabelle Macaraeg       |             2 | Midterm | Activity 1   |  36.00  |       50.00 |  10.00 |
|        1 | 25-01205   | Ysabelle Macaraeg       |             3 | Midterm | Midterm Exam |  85.00  |      100.00 |  30.00 |
+----------+------------+-------------------------+---------------+---------+--------------+---------+-------------+--------+
36 rows in set (0.009 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> SELECT * FROM vw_open_revision_requests ORDER BY requested_at;
+------------+-------------------+------------+-------+-----------------+---------------------------+--------------+---------------------+
| request_id | full_name         | title      | score | requested_points | reason                    | status       | requested_at        |
+------------+-------------------+------------+-------+-----------------+---------------------------+--------------+---------------------+
|          1 | Lorenzo Villaflor | Quiz 1     |  9.00 |            2.00 | Misread answer on item 5  | Pending      | 2026-10-14 22:38:53 |
|          2 | Kyla Mae Dimaculangan | Activity 1 | 41.00 |            NULL | Please re-check my total score | Under Review | 2026-10-14 22:38:53 |
+------------+-------------------+------------+-------+-----------------+---------------------------+--------------+---------------------+
2 rows in set (0.001 sec)
```

```
MariaDB [centralized_and_transparent_grading_system]> SELECT * FROM vw_grade_history ORDER BY logged_at DESC;
+--------+----------+-------------------------+----------------+-----------+-----------+-----------+---------------------+
| log_id | actor_id | action                  | entity_type    | entity_id | old_value | new_value | logged_at           |
+--------+----------+-------------------------+----------------+-----------+-----------+-----------+---------------------+
|      9 | P001     | Updated request status  | RevisionRequest | 3         | Pending   | Approved  | 2026-10-11 15:30:00 |
|     10 | P002     | Updated request status  | RevisionRequest | 4         | Pending   | Closed    | 2026-10-11 15:30:00 |
|     11 | P001     | Updated score           | StudentGrade   | 6         | 88/100    | 93/100    | 2026-10-11 15:30:00 |
|      8 | P001     | Updated request status  | RevisionRequest | 2         | Pending   | Under Review | 2026-10-11 08:50:00 |
+--------+----------+-------------------------+----------------+-----------+-----------+-----------+---------------------+
4 rows in set (0.002 sec)

MariaDB [centralized_and_transparent_grading_system]>
```

7. VB.NET System Implementation

Develop a working VB.NET front-end connected to the MySQL database. The interface must support the approved project scope and enforce relevant business rules.

7.1 Reports / Outputs

7.1 Reports / Outputs

Professor

- **Class Grade Sheet:** displays students' assessment scores, running grades, and academic status. It can be filtered and **exported as a PDF**.
- **Class Performance Summary (Analytics):** displays the class average, number of passing students, number of at-risk students, and highest grade.
- **Averages by Assessment Type:** displays average scores for quizzes, activities, examinations, and projects using progress bars.
- **Audit Log – Recent Changes:** records score updates and request status changes, including the affected student, item, date and time, and user responsible. It can be **exported as a PDF**.
- Digital Gradebook: displays persisted raw assessment scores, percentages and shared weighted running grades. It supports PDF export; legacy worksheet-only inputs with incompatible calculations are hidden.
- **Student Grade Record:** displays a selected student's assessment scores and allows professors to record scores and recalculate grades.
- **Clarification Requests:** displays students' grade clarification requests, including the assessment, current score, requested adjustment, reason, and request status.

Student

- **My Grades (Grade Report):** displays assessment scores, relative weights, running grades, and academic status. Students can filter grades by midterm, finals, or combined results and select a course.
- **Grade History:** displays recorded grade changes, including the date, assessment, old score, new score, person responsible, and reason.
- **My Performance Overview:** displays the current average, activities passed, open clarification requests, and highest score. Progress bars show performance in quizzes, activities, examinations, and projects.
- **Clarification Request Status:** displays submitted requests, including the course, assessment, requested action, and current status. It also shows the reviewer's response.
- Student Profile: a read-only dialog shows student ID, name, email, program, year, section, status, enrolled-class count, open-request count and current per-class grades and standing. Open requests include Pending and Under Review.
- Change Password: students can change their own password; teachers also have a Change Password button beside Log out. Both require the current password and use the existing hashing and audit service.

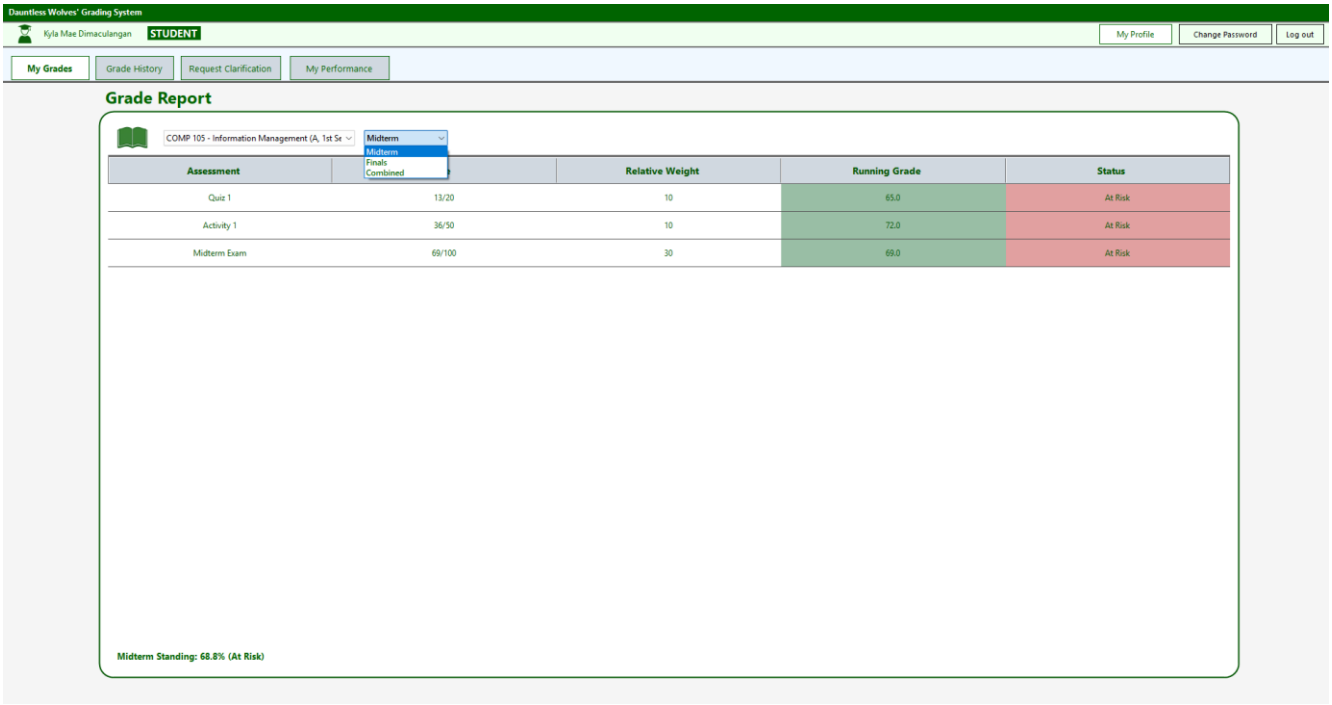
Admin

- Admin Dashboard: displays student, professor and class counts, open-request count and recent updates. The Active Classes caption reflects the available class count; there is no class active/inactive status field.

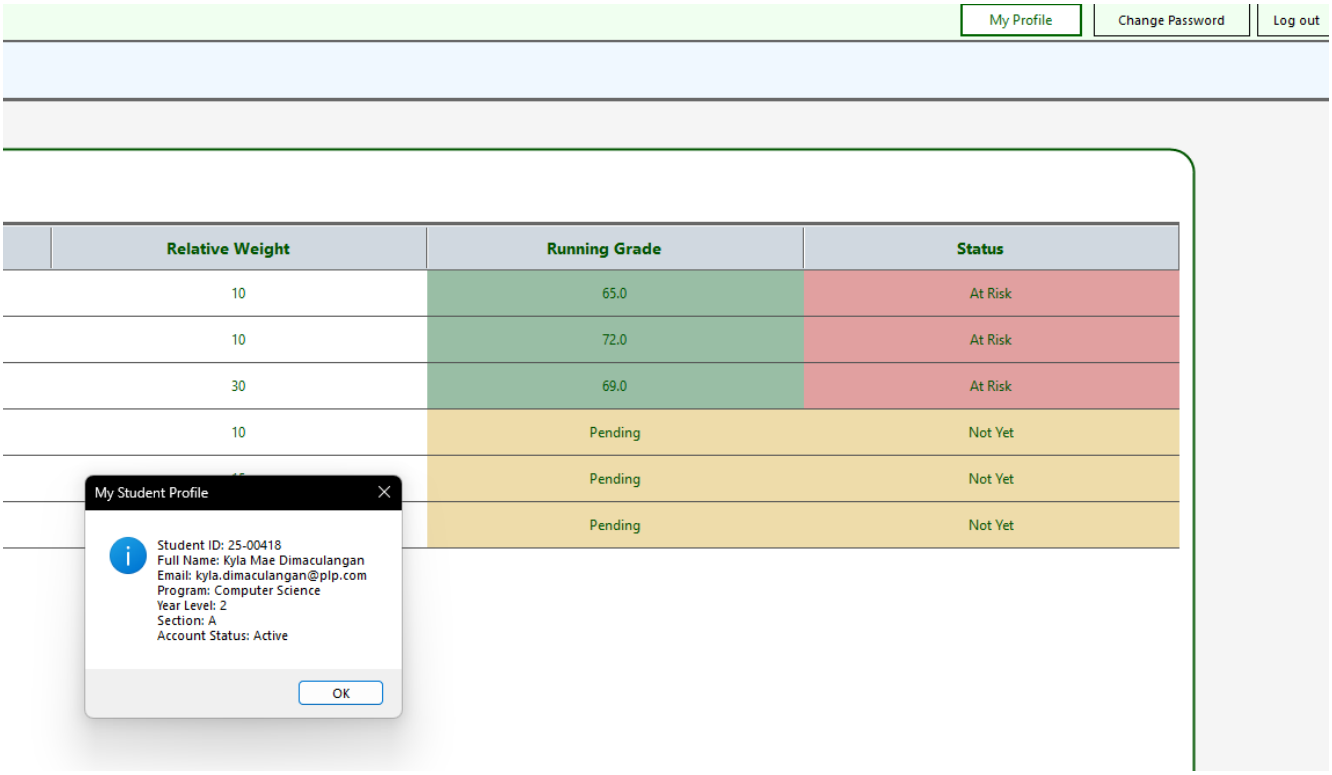
- **Student Records:** displays student IDs, full names, year levels, sections, programs, and account statuses. Administrators can add, edit, search, filter, and delete student records.
- **Professor Records:** displays professor names, employee IDs, departments, assigned classes, employment types, and account statuses. Administrators can add, edit, search, filter, and delete professor records.
- **Class/Subject Management:** allows administrators to create classes by entering the subject code, subject name, section, assigned professor, semester, and academic year.
- **Grade Entry:** allows administrators to select a student, subject, and assessment before entering and saving grades.
- Clarification Requests: read-only administrative overview with request and review dates, reasons, teacher responses, offering/status filters and CSV/PDF exports. Administrators do not decide these requests here.
- **Audit Logs:** records administrative and system activities, including the user responsible, action performed, affected entity, previous and updated values, date, time, and additional details. Records can be filtered and exported as CSV or PDF.

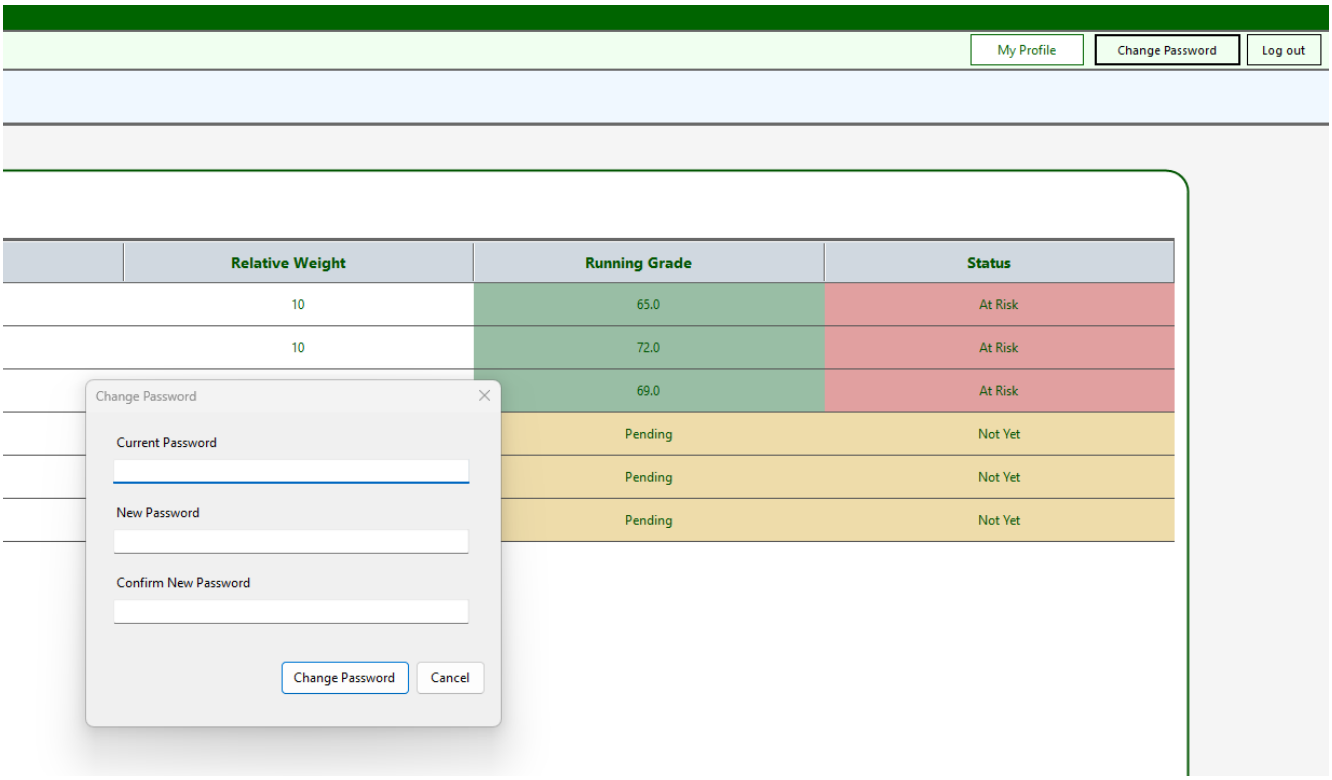
7.2 System Screenshots

STUDENT



My Grades – View grades for quizzes, activities, exams, and projects. Include Midterm, Finals, and Combined views, running grades, assessment weights, and academic standing.

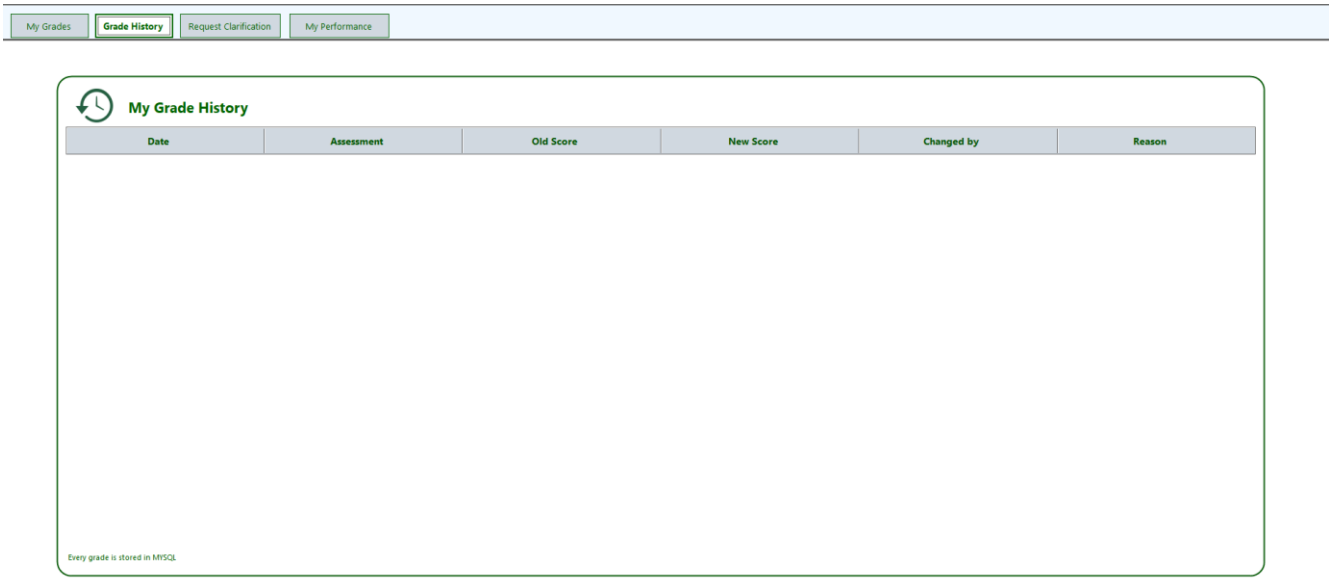




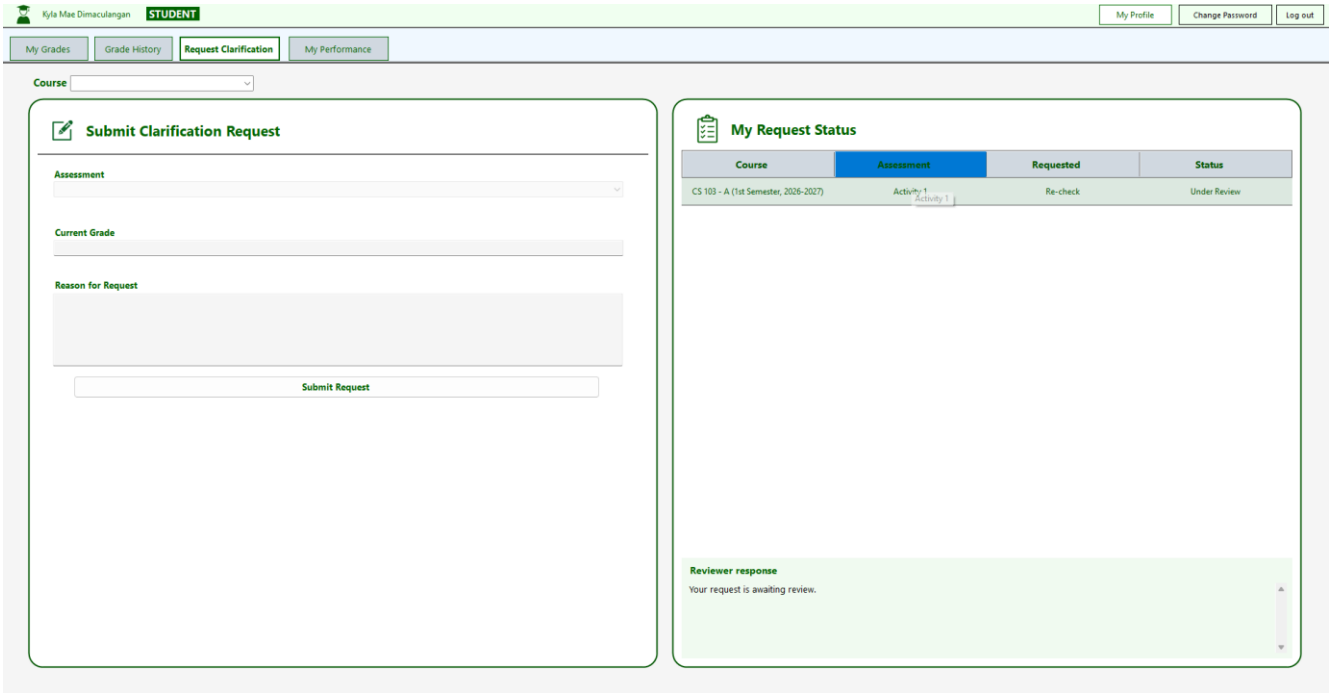
My Profile & Account Security – View student information and change the account password, with confirmation and validation.

Student profile: click My Profile to view personal details, enrolled classes, open requests and per-class standing. Click Refresh to reload current records, or Close/Escape to return. This form is read-only.

Professor password: click Change Password beside Log out; enter the current password, a new password of at least four characters, and matching confirmation. Click Change password. On success, use the new password at the next login. Cancel/Escape makes no change.



Grade History – View changes to grades, including the date, assessment, old score, new score, person who changed the grade, and reason.

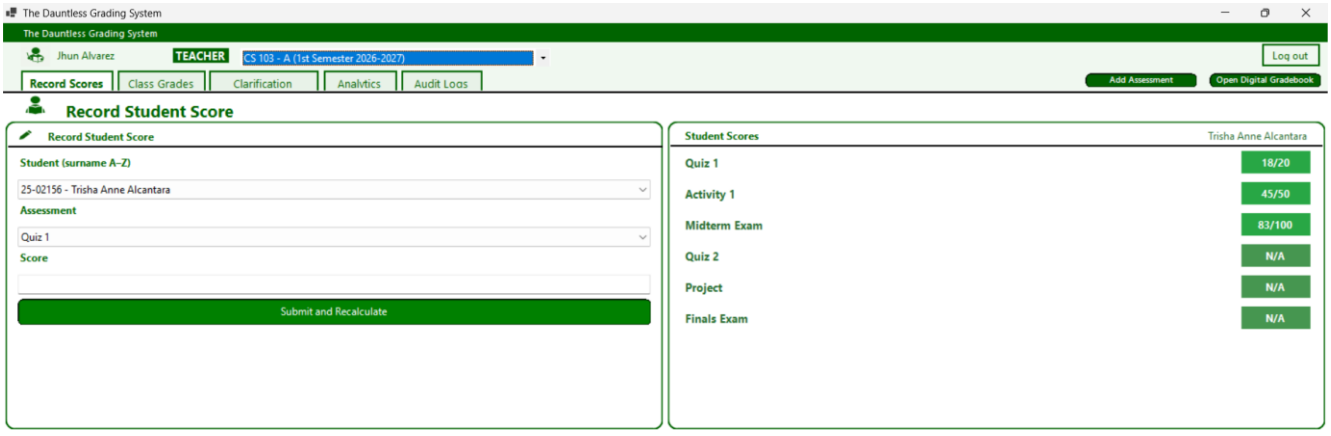


Request Clarification – Submit requests for grade rechecks, select a course and assessment, provide the current grade and reason, and track request status and reviewer responses.

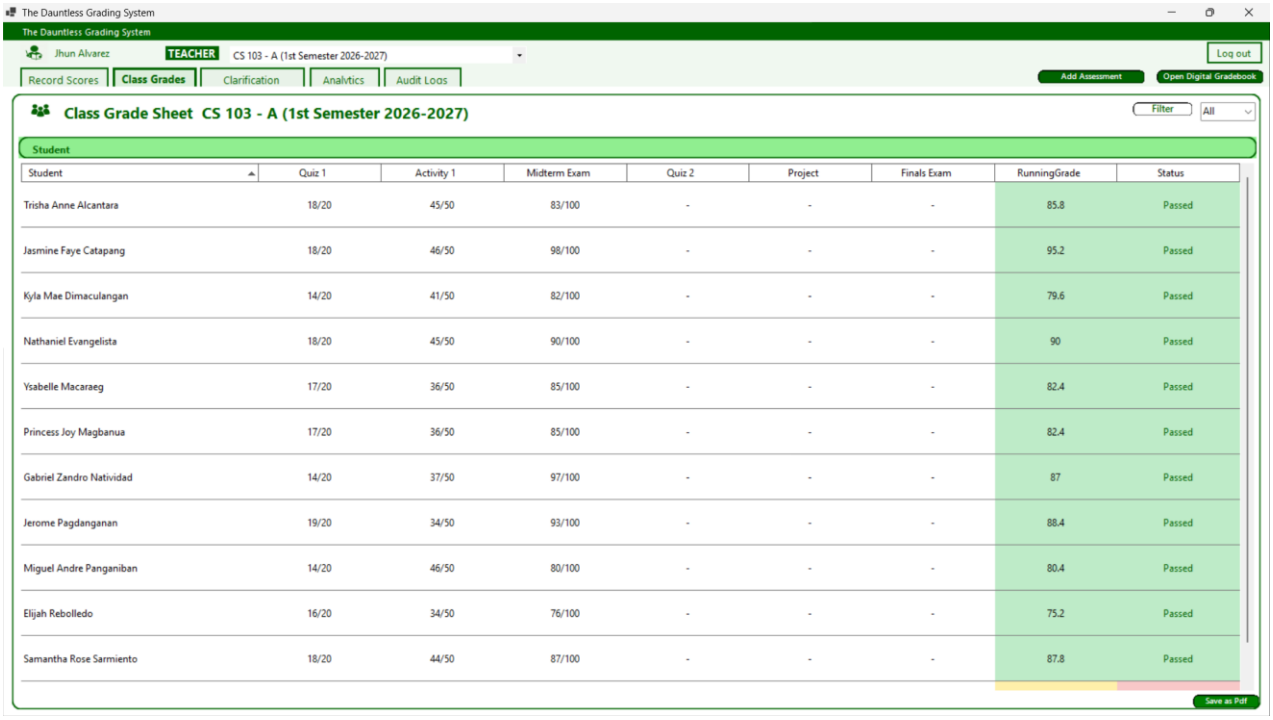


My Performance – Display the student's current average, activities passed, open requests, highest score, and performance percentages for quizzes, activities, exams, and projects.

PROFESSOR

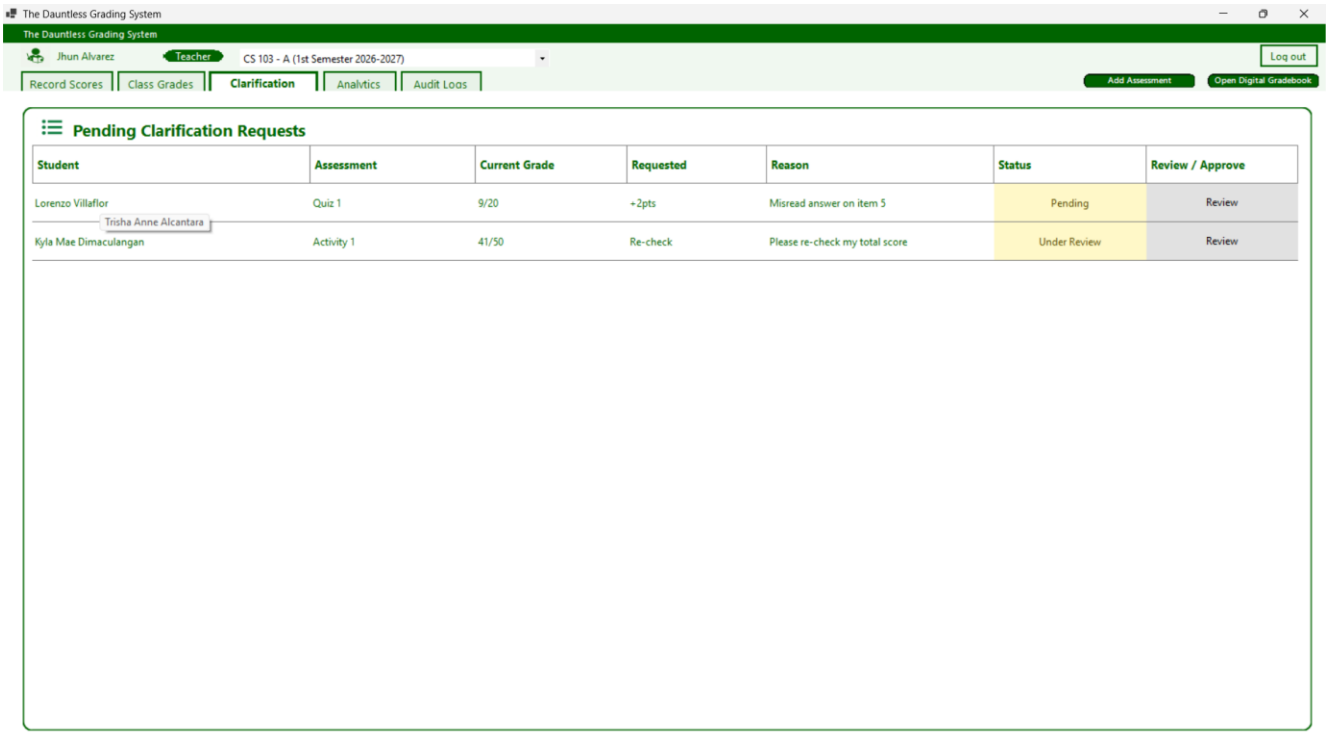


Record Student Score
Allows professors to select a student, choose an assessment, enter scores, and recalculate grades automatically.

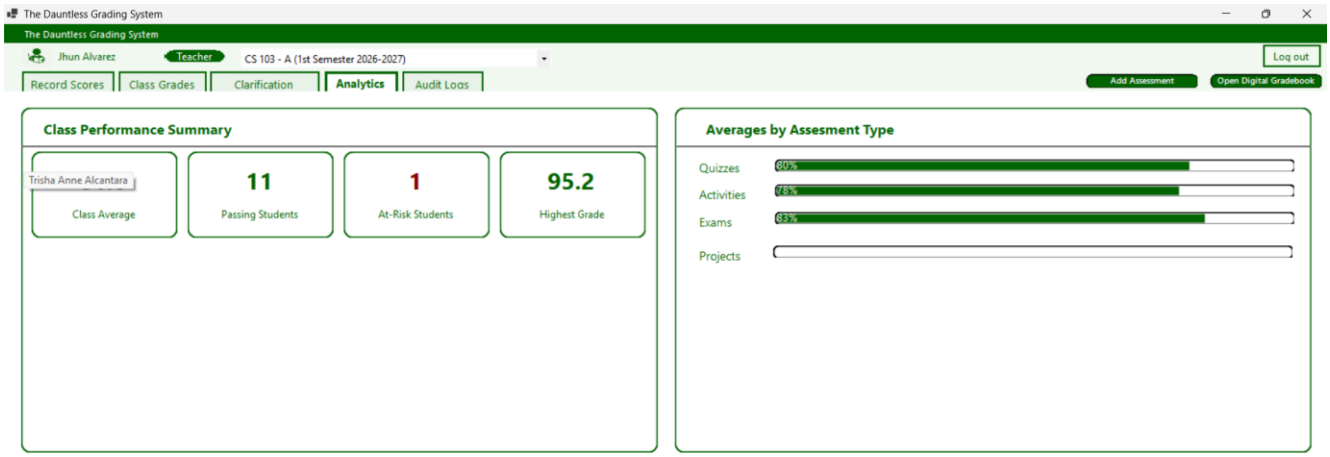


Class Grade Sheet

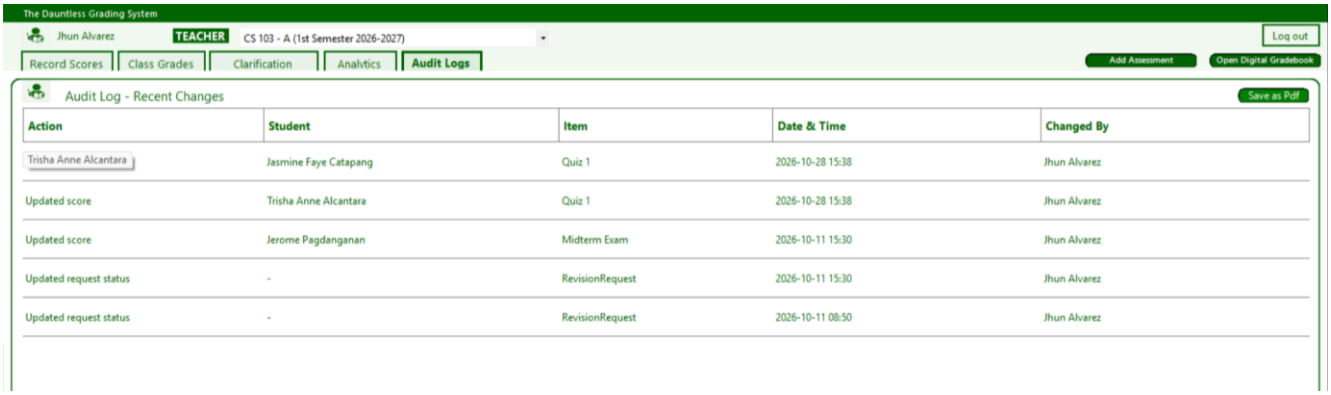
Displays students' assessment scores, running grades, and academic status. Professors can filter records and export the grade sheet as a PDF.



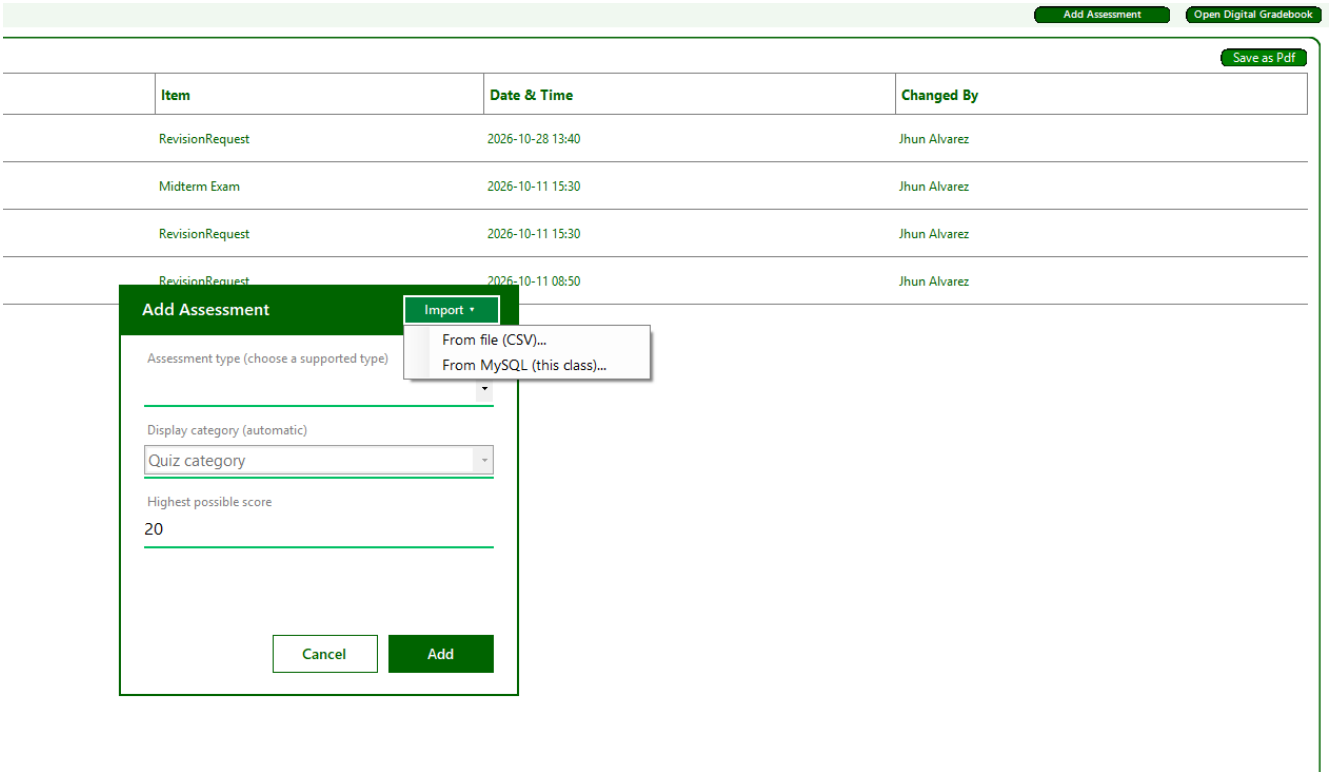
Pending Clarification Requests
Displays students' grade clarification requests, including assessment details, current grades, requested adjustments, reasons, and request statuses. Professors can review and approve requests.



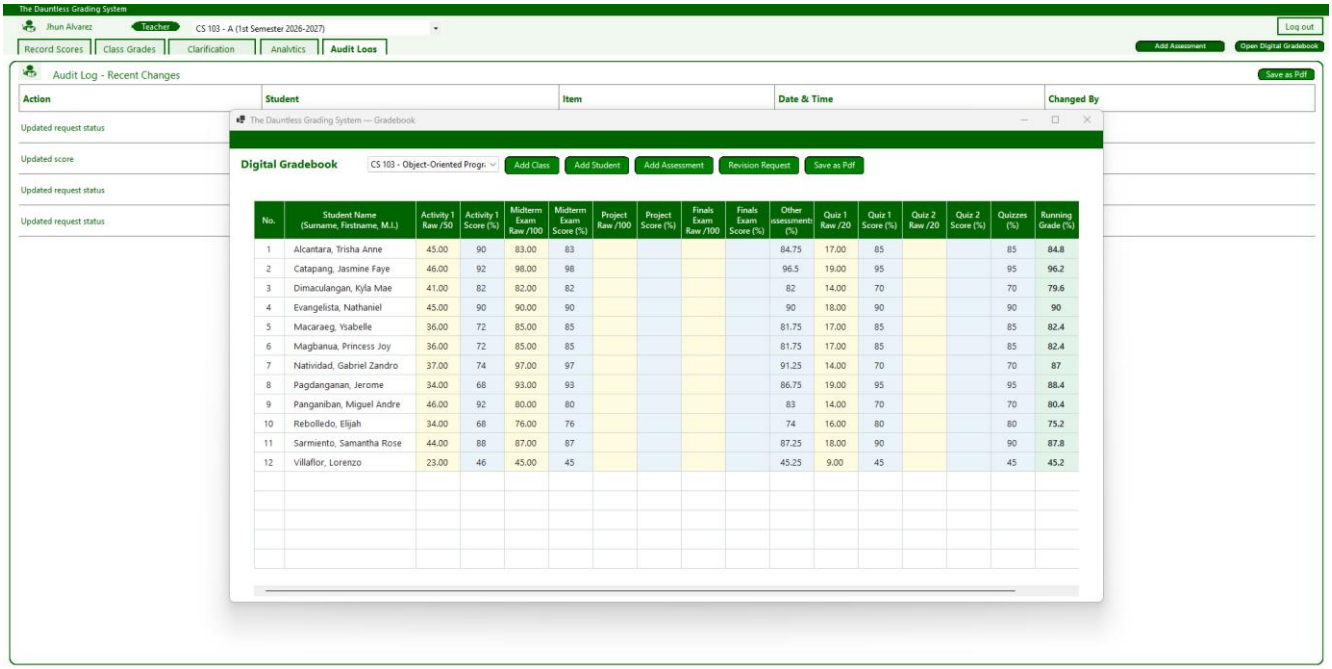
Analytics
Displays the class average, number of passing students, at-risk students, and highest grade. Progress bars summarize average performance by assessment type.



Audit Log
Records score updates and clarification request status changes, including the affected student, assessment, date and time, and responsible user. Records can be exported as a PDF.



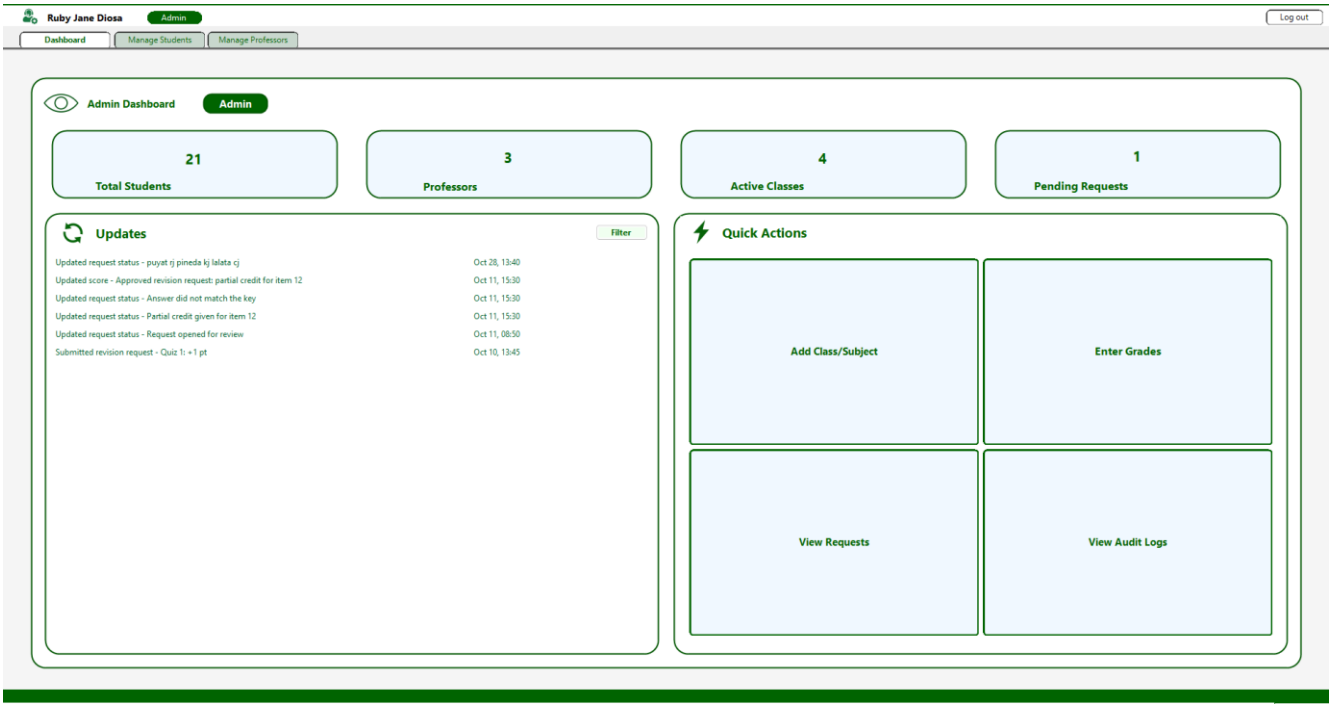
Add Assessment: create Quiz, Activity, Exam, Project or Attendance items with maximum score, Midterm/Finals period and relative weight. Assessment definitions can be imported from CSV; database refresh reloads existing configured records rather than creating an external integration.



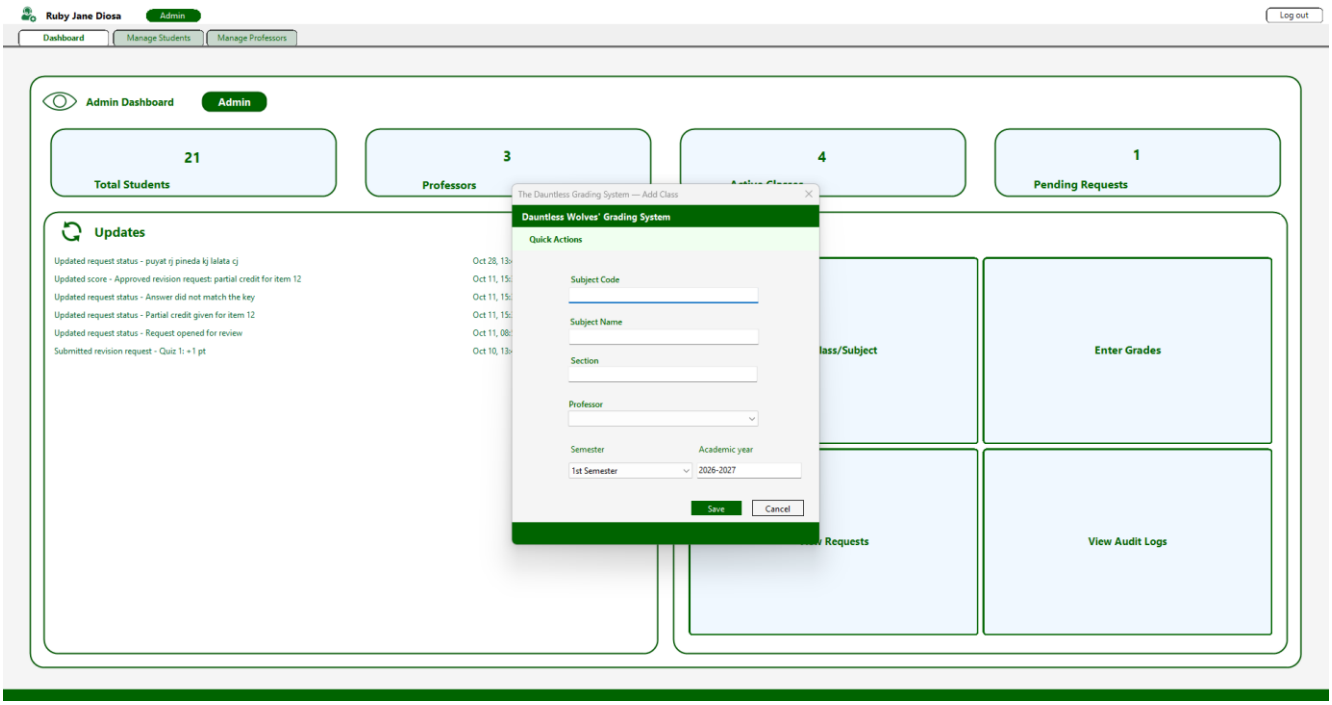
Digital Gradebook

Displays students' raw scores, percentage scores, and running grades in a spreadsheet-style layout. Professors can manage classes, students, and assessments, handle revision requests, and export the gradebook as a PDF.

ADMIN

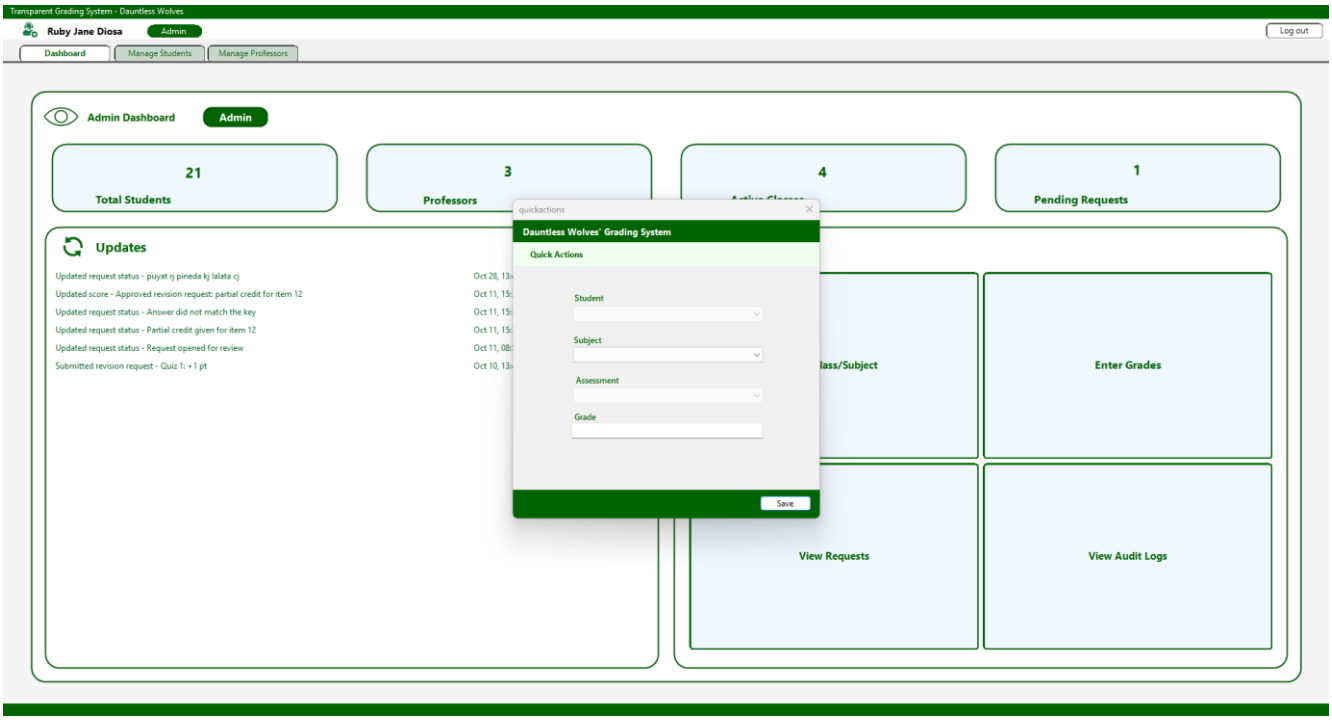


Admin Dashboard
Displays system statistics, recent updates, and quick actions for managing classes, grades, clarification requests, and audit logs.



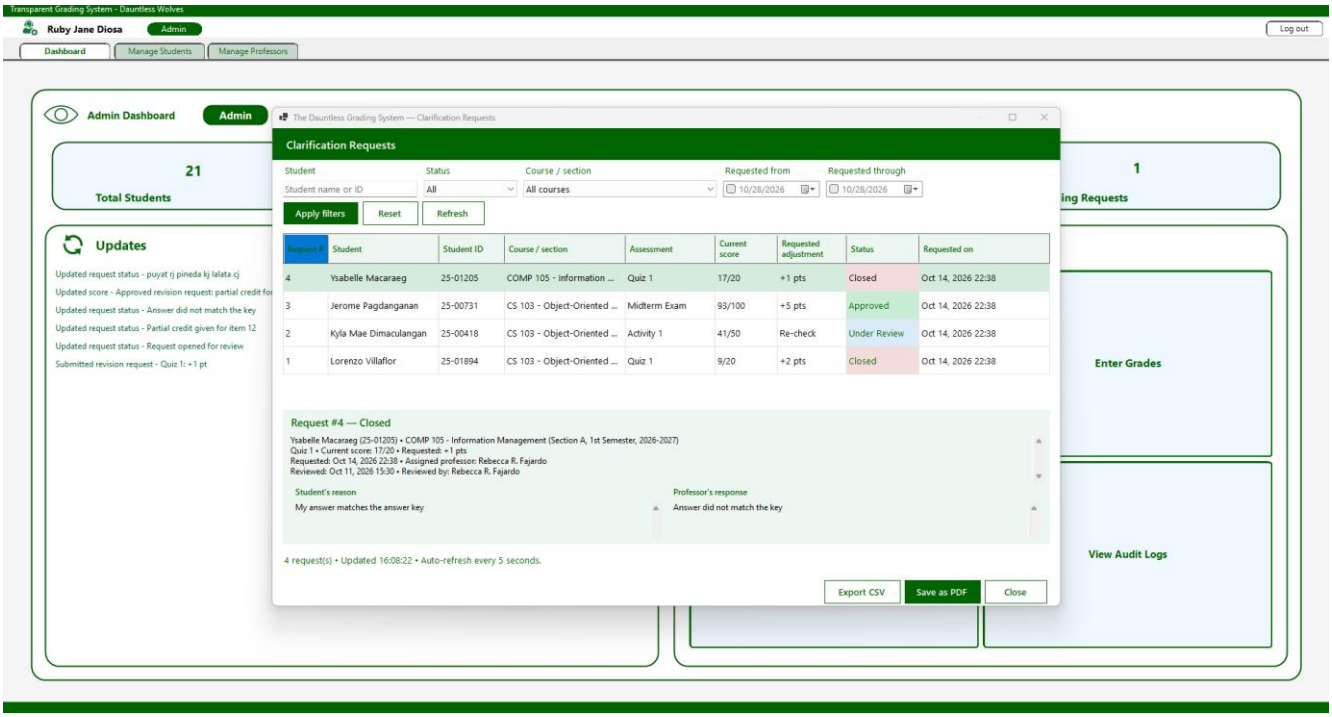
Enter Grade

Allows administrators to insert the grade of each student specific of the matching course with the ability to choose which semester.



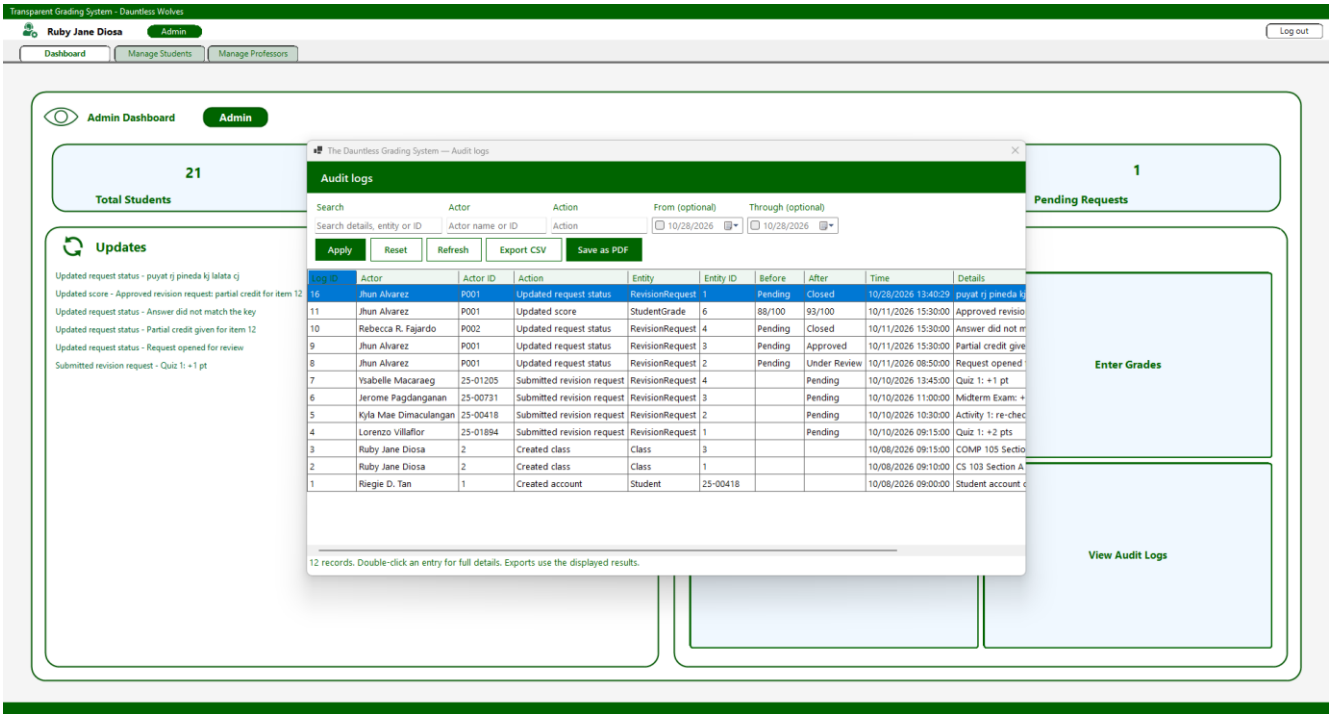
Add Class/Subject

Allows administrators to create classes by specifying subject details, sections, professors, semesters, and academic years.



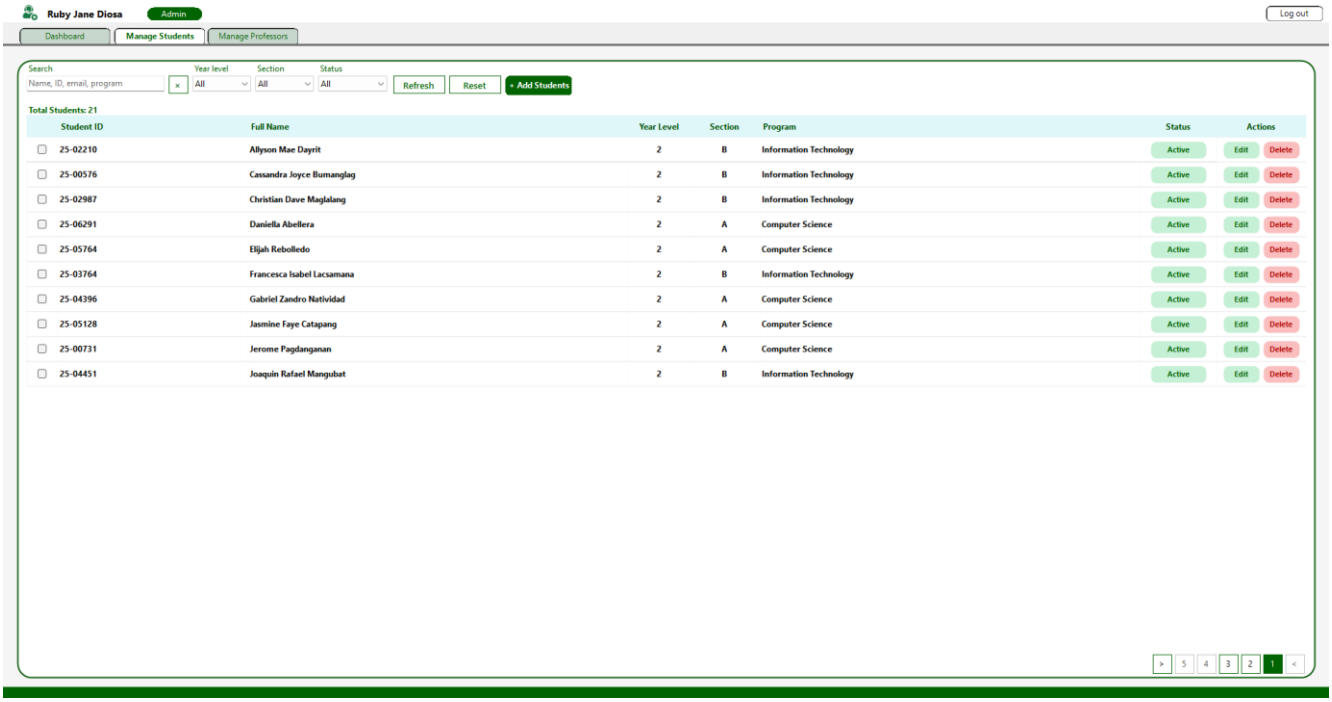
Clarification Requests

Displays students' grade clarification requests, including scores, requested adjustments, and request statuses.



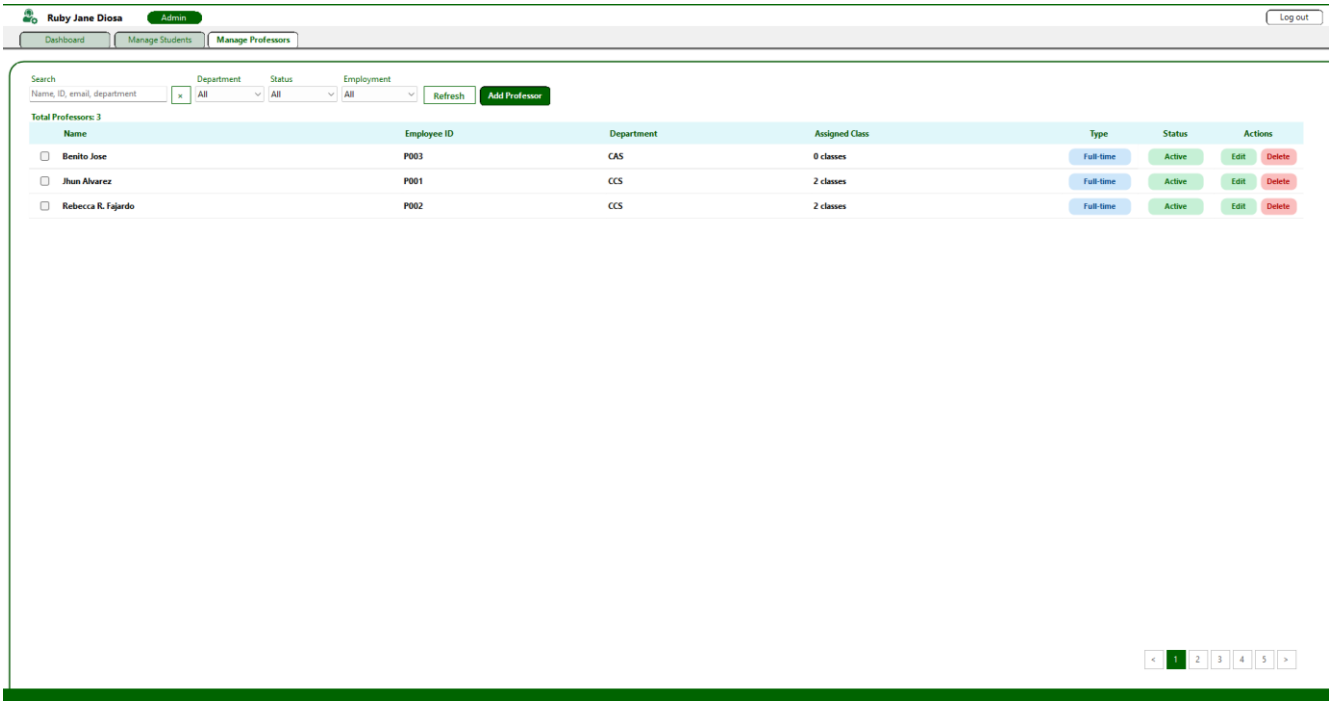
Audit Logs

Records system activities, including changes made, affected records, timestamps, and details. Administrators can also export logs to CSV or PDF.



Manage Students

Allows administrators to search, filter, add, edit, and delete student records.



Manage Professors
Allows administrators to manage professor information, departments, employment types, and assigned classes.

7.8 Form/Module Inventory

The inventory below identifies the actual forms and their database responsibilities. Shared services handle validation, transactions, authorization, exports and calculation rather than duplicating business rules in each form.

Form/Module	Purpose	Tables Used	Main Functions	User Role
frmlogin	Authenticate and route users	student, teacher, admin	Role-aware login; Enter submission; password verification	All
frmstudent	Student academic portal	enrollment, class, course, assessment, studentgrade, revisionrequest, auditlog	Own grades/history/performance; requests and responses	Student
StudentProfileForm	Read-only profile and summaries	student, enrollment, class, course, assessment, studentgrade, revisionrequest	Personal details; enrolled/open counts; per-class standing	Student

frmprof	Teacher portal	teacher, class, enrollment, assessment, studentgrade, revisionrequest, auditlog	Fast score entry; class sheet; analytics; reviews	Teacher
frmGradebook	Spreadsheet-style saved scores	class, course, student, enrollment, assessment, studentgrade, auditlog	Persisted edits; CSV imports; enrollment; PDF	Teacher
ProfessorPasswordForm	Own password change	teacher, auditlog	Current-password verification; hashing; transactional audit	Teacher
Admin / frmUserEditor	Account management	student, teacher, class, enrollment, revisionrequest, auditlog	Create/edit; status/employment; protected delete	Admin
quickactions / frmAddClass	Administrative class/grade operations	course, class, teacher, enrollment, assessment, studentgrade, auditlog	Create offering; authorized grade entry	Admin
AdminRequestsForm / AdminAuditForm	Oversight and reporting	revisionrequest, studentgrade, enrollment, class, course, student, teacher, admin, auditlog	Read-only requests; date/search filters; CSV/PDF	Admin

7.9 Validation and Test Evidence

Recorded validation: 211 backend integration checks passed against a disposable seeded database. A separate CSV validation run passed 48 checks. The student-profile/professor-password suite passed 21 targeted checks. These are separate runs, not a claim of exhaustive coverage. The application builds with zero warnings/errors in the recorded build. Windows interaction harnesses compile; their compilation is not proof of runtime success.

Test area	Expected result	Recorded status	Evidence / follow-up
Grade validation and ownership	Reject invalid scores and unauthorized writes.	Backend cases passed	validation/README.md; professor-services harness
Import and batch rollback	Failed supported transactional batch leaves no partial data.	Backend cases passed	Disposable failure-injection checks
Student profile isolation	Own details only; correct counts and weighted grades.	21-check suite includes this area	validation/profile-password.vbproj
Professor password	Wrong current password rejected; new password works; audit retained.	21-check suite includes this area	validation/profile-password.vbproj

CSV samples	Valid files accepted; intentional invalid files rejected.	48 checks passed	docs/test-csv/README.md
Windows layouts and keyboard	No clipping; Enter/Escape and window behavior correct.	Group verification required	Capture actual results at 100%/125% scaling
Final PDF print/export	Readable complete report and saved destination file.	Group verification required	Run on Windows with Microsoft Print to PDF

Attach the group's actual Windows screenshots and execution dates in Appendix F. Do not mark unexecuted cases Passed. No respondent count, survey result, measured speed improvement or formal approval is supplied in the source documents.

8. User Manual

8.1 System Overview

The Transparent and Centralized Grading Management System for Pamantasan ng Lungsod ng Pasig (DAUNTLESS_WOLVES_PROF) is a desktop application developed in VB.NET WinForms paired with a MySQL/MariaDB database. It helps in transforming manual computation errors and lack of academic transparency by replacing manual spreadsheets with an automated, auditable, and role-based grading infrastructure.

The system serves three primary user groups:

- System Administrators: Responsible for institutional setup, account management (CRUD for students and professors), class assignments, and overall system audit monitoring.
- Faculty / Professors: Responsible for defining class assessment weightings, recording student raw scores, reviewing automated running grades, processing grade clarification requests, viewing performance analytics, and exporting official grade sheets.
- Enrolled Students: Responsible for tracking real-time academic performance across enrolled subjects, reviewing historical grade change logs, and submitting formal score revision requests with written justifications.

8.2 System Requirements

Operating system: Windows 10/11 with the .NET 8 Windows Desktop Runtime for a framework-dependent build. Development: Visual Studio 2022 with .NET desktop development and a .NET 8-compatible SDK. Database: a compatible MySQL/MariaDB server; the supplied dump identifies MariaDB 10.4.32. Restore the MySql.Data 26.7.0 NuGet dependency. PDF export requires Microsoft Print to PDF. Configure dbconfig.json beside the executable.

Hardware: use a Windows computer capable of running these components, with enough storage for the application, database and backups. No measured minimum RAM, CPU or concurrent-user capacity was established. A 1366×768 display is a practical review target; verify both 100% and 125% scaling.

8.3 Installation / Setup

Requirements: Windows 10/11, Visual Studio 2022, and MySQL/MariaDB (XAMPP).

A. Set up the database

1. Start MySQL (XAMPP Control Panel or Windows Services).
2. Open MySQL shell and log in as root.

Create centralized_and_transparent_grading_system, then select it before importing the supplied SQL dump. Use CREATE DATABASE IF NOT EXISTS centralized_and_transparent_grading_system;

4. Import the supplied complete SQL dump. Import separate sample_data.sql only if the submission actually provides one; do not import the same sample rows twice.

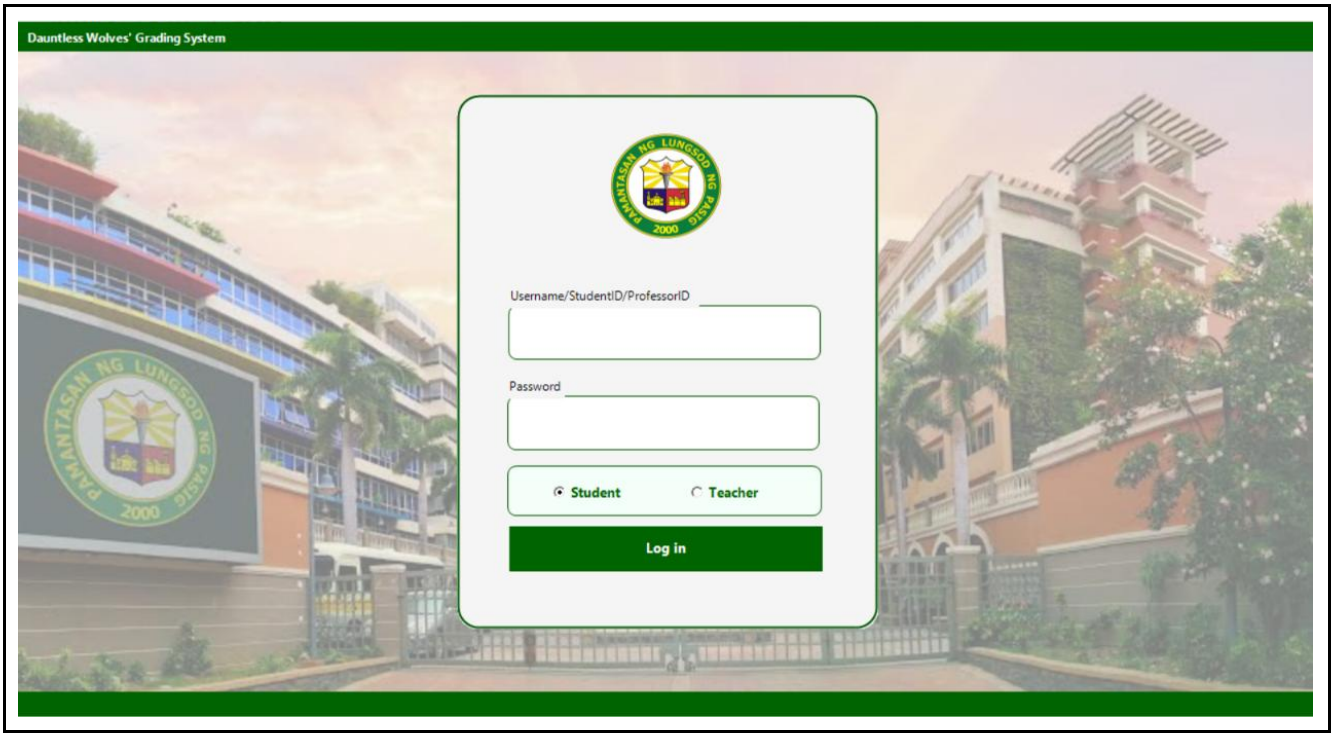
B. Connect the application

1. Create or edit dbconfig.json beside the application executable. Use the existing file if provided; no separate example file is required.
2. Set Host, Port, Database, User and Password to the actual server values. Database must be centralized_and_transparent_grading_system if using the supplied dump. Use a restricted database account for deployment; do not assume root with a blank password is appropriate.

C. Run the application

1. Open the .sln file in Visual Studio 2022.
2. Right-click the solution and choose Restore NuGet Packages.
3. Make sure frmlogin is the startup form, then press F5.

8.4 Login / Starting the System



1. Run the built application executable (normally DAUNTLESS WOLVES PROF.exe) from its output folder, or start the solution through Visual Studio.
2. The **User Login Portal** (frmlogin.vb) will open.
3. Enter the assigned student ID, teacher ID or email. Select Student or Teacher as appropriate; administrator email login is available with either role selection. Use a registered account with an initialized password.
4. Enter your account **Password** into the txtPassword field.
5. Click Log In or press Enter in the username or password field. Invalid credentials or inactive student/teacher accounts must not open the portal.

8.5 Main Menu / Navigation

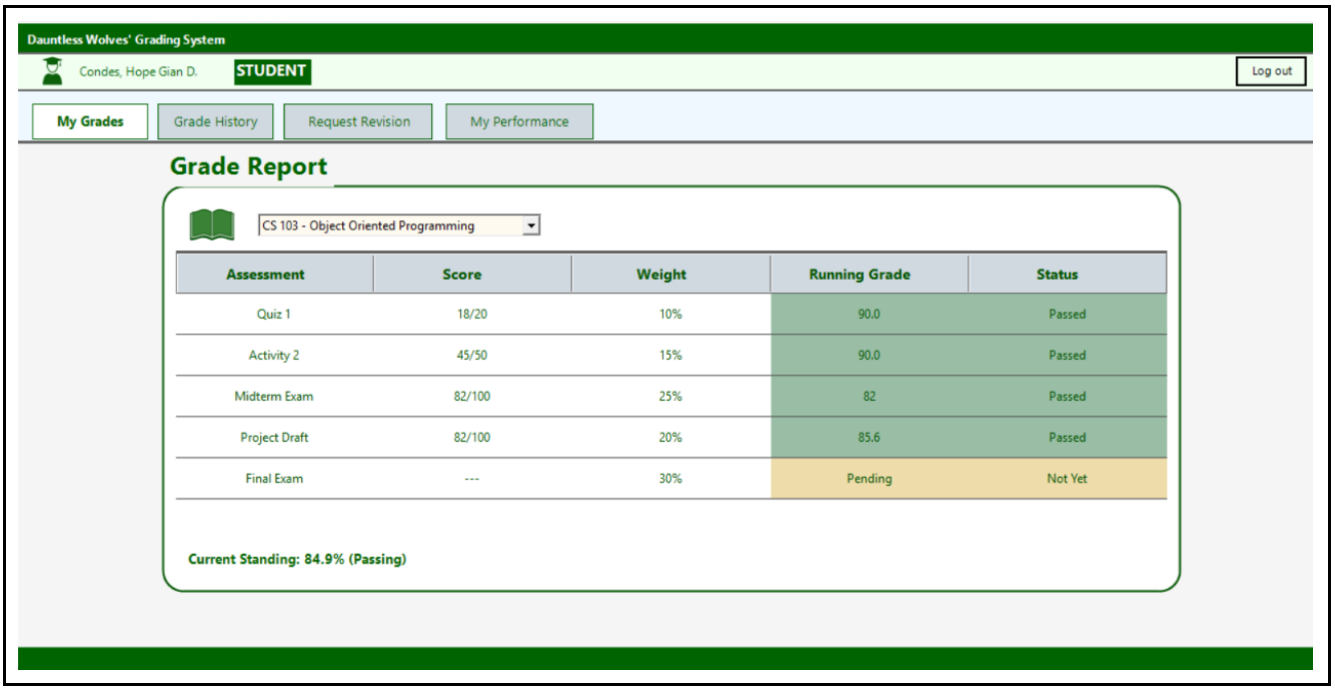


Figure 8.5.A Student Main Dashboard

1. My Grades - this form exhibits the current running grade of the student, it also shows the accomplished and pending activities of the student. Status of the assessment is also indicated here.
2. Grade History - Transparent timeline recording every score entry and modification, old vs. new values, teacher notes, and timestamps
3. Request Clarification: select a graded assessment, enter a reason and submit a formal review request. Free-text adjustments do not automatically change a score. The assigned teacher supplies or approves the correction.
4. My performance : It demonstrates the overall performance of the student in his/her current semester.

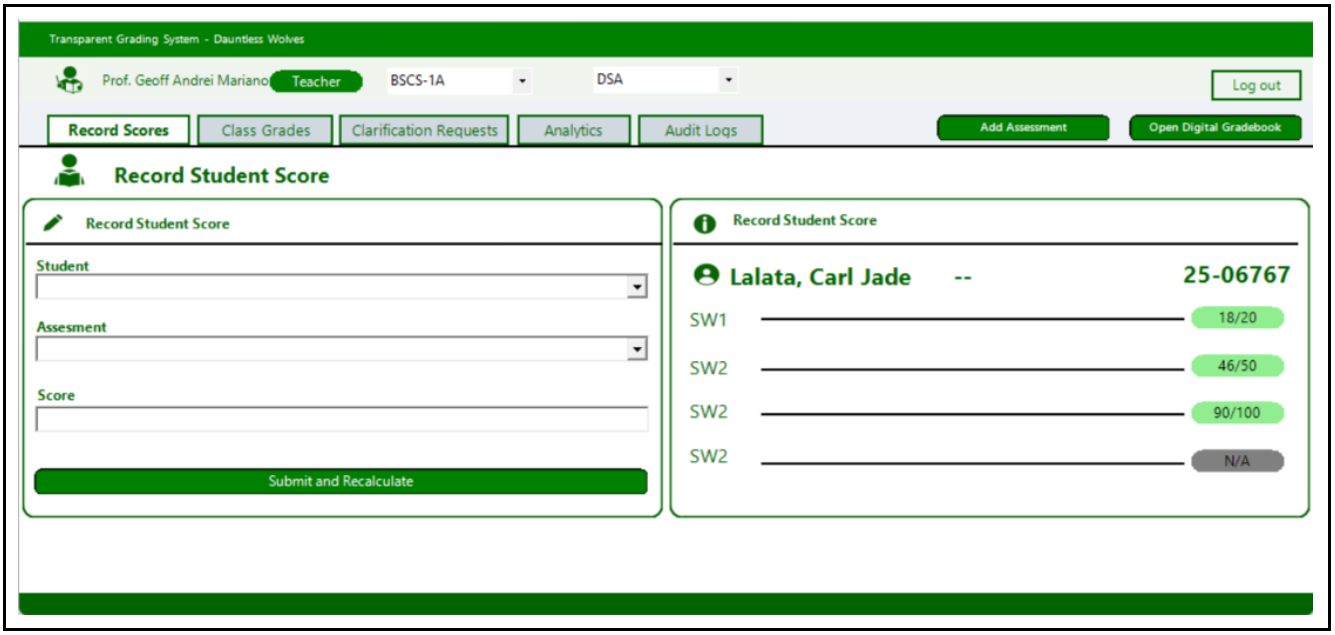


Figure 8.5.B Professors Main Dashboard

1. Shows the complete class list with automatically calculated grades, visual pass/fail indicators, options to filter by term, and the ability to export results as a PDF
2. Class Grades:Shows the complete class list with automatically calculated grades, visual pass/fail indicators, options to filter by term, and the ability to export results as a PDF
3. Revision/Clarification Requests : Queue to evaluate student dispute requests, view written justifications, and approve or reject requested score changes.
4. Class Performance Analytics: Visual graphs and progress bars showing class averages, top performers, passing distribution, and assessment breakdowns.
5. Audit Logs: Maintains a detailed, timestamped log of all grade entries, modifications, and revision approvals made by faculty to ensure data integrity.
6. Digital Gradebook: Displays a spreadsheet-style, full-screen grid of student scores across all course components, with options to export the data as a PDF.

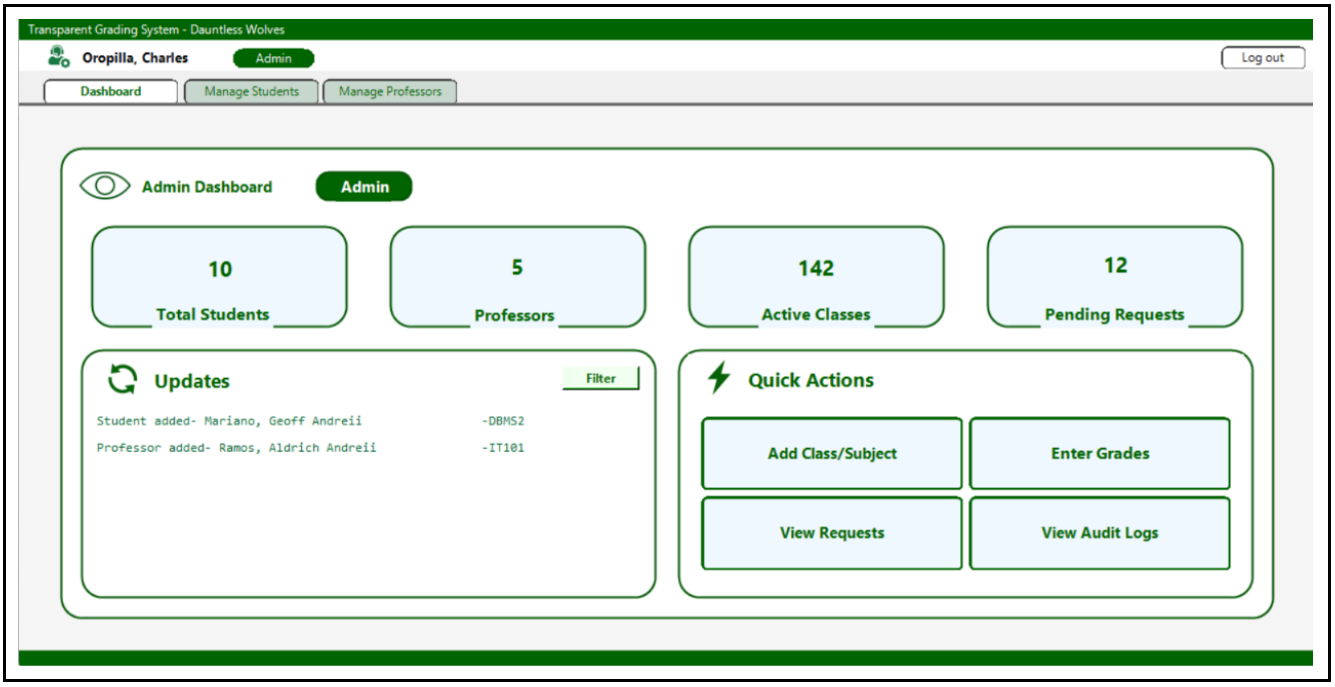


Figure 8.5.C Admin Main Dashboard

1. Dashboard: student, professor and class counters, open-request count and recent updates. No separate system-health metrics are implemented.
2. Manage Students: Directory interface to add, edit, filter, or deactivate student accounts.
3. Manage Professors: Directory interface to manage faculty listings and assign academic department parameters.
4. Class & Section Offering: Module to create subject sections and link assigned professors.
5. Audit History: Comprehensive log viewer recording administrative actions across the platform.

8.6 Backup / Restore or Data Maintenance

1. Database Backup Procedure

1. Open a terminal or administrative command prompt on the database server host.
2. Run mysqldump to generate a timestamped SQL backup file:
3. Bash

```
mysqldump -u root -p centralized_and_transparent_grading_system > centralized_and_transparent_grading_system.sql
```

```
backup_grading_system_%date:~10,4%%date:~4,2%%date:~7,2%.sql
```

4. Alternatively, in phpMyAdmin, select centralized_and_transparent_grading_system → click Export → select Quick Export Method → click Go.

2. Database Restore Procedure

1. Create or clear the target database instance.
2. Execute the import command via terminal:
3. Bash

```
mysql -u root -p transparent_grading_system < backup_grading_system_YYYYMMDD.sql
```

4. Or import the backup `.sql` file using phpMyAdmin's **Import** tab.

9.9 Logout / Exit

1. Logging Out of an Active Session

1. Click the Logout button located on the top navigation bar of any active portal.
2. The application clears the local AppSession user information. It does not delete the account or change its password. The portal closes and returns to the login window.
3. The active portal form closes, and the system redirects back to the User Login Portal.

10. Conclusion and Recommendations

10.1 Project Conclusion

The Transparent and Centralized Grading Management System brings account management, enrollment, assessment definition, score recording and clarification requests into a database-backed Windows application. Its role-based portals organize administrative, teacher and student responsibilities while providing access to current academic records.

A shared weighted calculator supports consistent standing across grading screens. Input validation, ownership checks and transactions help protect supported operations from invalid input and partial saves. Audit entries and teacher review responses make score corrections more traceable without giving students direct grade-editing authority. CSV imports, faster score entry and report exports support routine record management.

Development included repeated feedback and regression checks addressing calculation inconsistencies, editing errors, refresh behavior and interface problems. Recorded backend results support the tested workflows, while Windows runtime, usability, printing and larger-scale deployment still require appropriate verification. The

project provides a practical foundation for organized and transparent grade management, with further improvements possible in security, accessibility, backup and integration.

10.2 Recommendations / Future Enhancements

The following enhancements are recommended beyond the current project scope. Their inclusion here does not imply they are implemented or evaluated:

1. Cloud / Web Migration: Re-architect the client application from a WinForms desktop environment to a cloud-hosted web framework (e.g., ASP.NET Core Blazor or React with a RESTful Web API backend) to allow multi-device browser access without local software installation.
2. Automated Push & Email Notifications: Integrate an automated notification engine (using SMTP or push services) to notify students immediately via email or mobile alerts when new grades are posted or revision requests are resolved.
3. LMS and University SIS Integration: Develop API connectors to sync class rosters, enrollment records, and final term marks directly with the university Registrar's Student Information System (SIS) and Learning Management Systems (e.g., Canvas or Moodle).
4. Native Mobile Application: Build Android and iOS companion apps using MAUI or React Native to give students quick mobile access to running grades and dispute updates.
5. Predictive Performance Analytics: Incorporate machine learning models to analyze early-term assessment trends, automatically triggering early intervention alerts for students at risk of academic failure.

11. References

Condes, H. G., Lalata, C. J., Mariano, G. A., Oropilla, C., & Puyat, R. J. (2026, August 23). Transparent and Centralized Grading Management System [Unpublished project concept paper]. CS102 – Object Oriented Programming with Database.

Dauntless Wolves. (2026). The Dauntless Grading System [VB.NET source code, SQL schema and validation documentation]. https://github.com/CoffeeIsThere/dauntless-wolves-re-re

Microsoft. (n.d.). Windows Forms overview. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/

Microsoft. (n.d.). Rfc2898DeriveBytes class. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.rfc2898derivebytes

Oracle. (n.d.). MySQL Connector/NET Developer Guide. https://dev.mysql.com/doc/connector-net/en/

MariaDB. (n.d.). Foreign keys. MariaDB Documentation. https://mariadb.com/docs/server/ha-and-performance/optimization-and-tuning/optimization-and-indexes/foreign-keys

12. Appendices

☐ Appendix A – Approved Concept Paper / Revision History

The concept paper supplied with this documentation is dated August 23, 2026. It identifies spreadsheet fragmentation, delayed feedback, overwritten grades, difficult manual interpretation, audit logging and revision requests as the initial project concerns. The implemented scope is described in Sections 1 and 7; proposed home access, immutable history and predictive trends must not be assumed to be implemented features.

☐ Appendix B – Complete ERD

See the ERD figure in Section 3.1 and the entity explanations in Section 3.2. Validate all labels in the embedded image against the final dictionary, especially teacher.employment_type.

☐ Appendix C – Normalization Worksheets

See the worksheets in Sections 4.1–4.4 and the accompanying dependency explanation. The prose documents the enrollment–assessment candidate key and intentional running-grade cache; older image labels should be checked before submission.

☐ Appendix D – Complete SQL Script (.sql)

Attach the supplied update3_centralized_and_transparent_grading_system.sql as a separate electronic file. It provides the actual ten-table schema and sample inserts; do not replace it with screenshots alone.

☐ Appendix E – Selected VB.NET Source Code / Repository Submission Details

Repository: https://github.com/CoffeeIsThere/dauntless-wolves-re-re . Source modules include Services/GradeCalculator.vb, GradeService.vb, RevisionRequestService.vb, AuthService.vb and Core/DatabaseHelper.vb. The profile and professor-password additions were merged to main through pull request #8. Submit the final agreed revision and retain its commit identifier.

☐ Appendix F – Additional Test Evidence

See Section 7.9 and validation/README.md. Add genuine Windows execution screenshots, actual dates, tester names and observed results. No results or signatures should be inferred from the test instructions.

☐ Appendix G – Additional User Manual Screenshots

Existing interface screenshots are retained in Section 7.2. Capture updated Student Profile and Professor Change Password screens on Windows if they are included in the submitted build.

☐ Appendix H – Other Instructor-Required Evidence

Professor remarks, approval/signatures, individual grades and any institution-specific forms must be completed by the responsible people. They are not generated as project evidence here.

Final Submission Checklist

☐ Concept Paper is complete and approved/revised.

☐ Business Rules are clear and reflected in the ERD.

☐ ERD matches the final normalized tables.

☐ Normalization process shows 1NF, 2NF, and 3NF—not only the final answer.

☐ Data Dictionary matches the MySQL database.

☐ MySQL database contains correct constraints and sufficient sample data.

☐ VB.NET system is connected to the submitted MySQL database.

☐ CRUD, validation, and required project functions are working.

☐ Test Cases include expected and actual results with evidence.

☐ User Manual is understandable to a first-time user.

☐ All screenshots are readable and captioned.

☐ All files use the required naming convention and are submitted in the required format.

REQUIRED Electronic Submission Package

Recommended folder structure: Folder name: Group # and Title

```
01_Documentation/
  Final_Project_Documentation.docx
  Final_Project_Documentation.pdf
02_Database/
  database.sql
  sample_data.sql
03_VBNet_System/
  [Visual Studio solution/project files]
04_Test_Evidence/
  [screenshots and test files]
05_User_Manual/
  User_Manual.pdf

06_Grade/
  Indi_score.xlsx
```